

Verizon / IBM WxO Workshop

Langflow Bootcamp

Table of Contents

<i>Langflow Bootcamp</i>	1
<i>How to Read This Document</i>	3
Screenshots.....	3
Formatting.....	3
<i>Set Up Workshop Environment</i>	4
Create an Astra DB Vector Database	4
Create a Collection.....	6
Store Your Token and Keys	7
<i>Exercise 1: Hello World Chatbot</i>	9
Connect the Dots.....	11
<i>Exercise 2: Document Ingestion with Metadata Enrichment</i>	14
Enhanced RAG Ingest Overview.....	14
1: File Component.....	15
2: Batch Run	16
3: Batch Run	16
4: Split Text.....	17
5: Batch Run	18
6: DataFrame Operations.....	18
7: Astra DB.....	19
8: IBM watsonx.ai	19
9: IBM watsonx.ai	19
10: IBM watsonx.ai Embeddings.....	20
Run Your Flow!.....	20
<i>Exercise 3: Enriched Data Retriever Flow</i>	21
Retriever Flow Overview.....	21
1: Chat Input	21
2: IBM watsonx.ai	21
3: Astra DB.....	22

4: IBM watsonx.ai Embeddings.....	22
5: Parser	23
6: Prompt Template.....	23
7: IBM watsonx.ai	24
8: Chat Output	24
Ask Some Questions	24
Exercise 4: Agentic Flow with Tool Calling	25
Tool Calling Agent Overview.....	25
1: Agent.....	26
2/3/4 : Astra DB.....	27
2: Astra DB Promotions	28
3: Astra DB System and Tools.....	28
4: Astra DB Account Support.....	29
5: IBM watsonx.ai	29
6: IBM watsonx.ai Embeddings.....	29
7: Text Input.....	30
8: Text Input.....	30
9: Text Output.....	30
Test your Agent.....	30
Exercise 5: Use Your Agent in the Retriever Flow	31
Agent RAG Retriever Flow Overview	32
1: Run Flow.....	32
Test Your Flow.....	32
Exercise 6: Multi-Agent Flow with MCP Tool.....	33
MCP Tool Flow Overview.....	33
1: Agent.....	34
2: MCP Tools.....	35
3: Calculator.....	37
4: IBM watsonx.ai	37
5: Chat Input & 6: Chat Output.....	38
Appendix A: Document Queries	38
Appendix B: Local Langflow Install.....	39
Langflow Desktop	39
Windows C++ Build Tools	39
MacOS xcode.....	40
OSS Langflow	40
MacOS	41
Windows	41

Welcome Verizon Team!

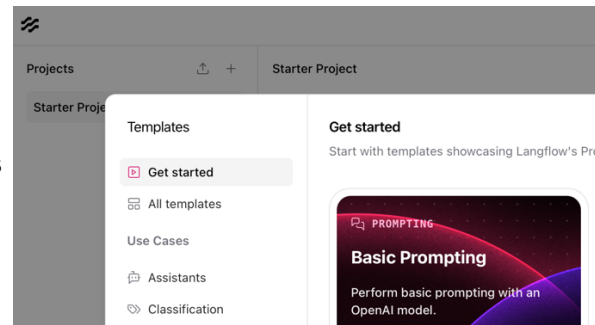
We're looking forward to a fun hands-on workshop with you. Below are resources and instructions you'll want to follow to get the most out of this workshop.

How to Read This Document

The intention of this document is to provide you with step-by-step instructions for completing the exercises. This workshop builds skills and understanding of the Langflow platform and is meant to be completed in order. Here are some formatting and highlighting concepts to be aware of so you can complete each exercise successfully.

Screenshots

Throughout the document you will see many screenshots indenting to show examples of how to configure your flows. Colored circles or callouts intend to show you specific areas to click or fill in.



Formatting

- `Gray monospace font` is used for names of components or the text of a button to click
- **Red** or **Purple** monospace font is used for data types and connector colors in Langflow
- `Orange monospace font` is text that you will either copy / paste from this document into your Langflow work, or the value of a parameter you will select from a dropdown.
- **GREEN MONOSPACE FONT** indicates a [global variable](#) is being used
- **Yellow highlighted text** is direction to pay special attention or to perform a critical action

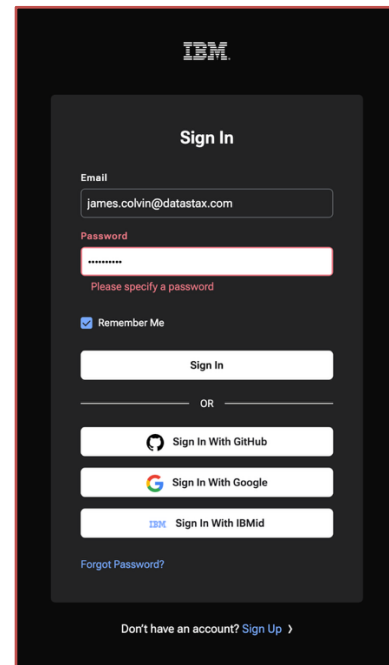
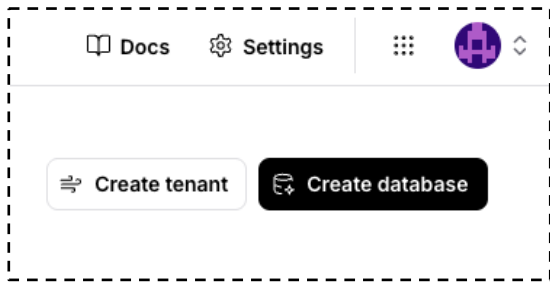
Set Up Workshop Environment

Create an Astra DB Vector Database

Sign up for a free DataStax Astra DB account directly from the [DataStax Astra portal](#).

Click **Sign Up** and create an account with your Verizon email address

Create your Astra DB database by Click on **Create database** on the top right hand menu. If you're signing up brand new you will be taken directly to the create database page.



1. Database Configuration:

- **Serverless (vector)**
- **Database name:** verizon_workshop
- **Provider:** Amazon Web Services
- **Region:** us-east-2

A screenshot of the 'Create database' form in the DataStax Astra DB console. The form is titled 'Create database' and has an 'ESC' button in the top right corner. It features two tabs: 'Serverless (vector)' (selected) and 'Serverless (non-vector)'. The 'Serverless (vector)' tab is described as 'An all-in-one database solution, optimized for Vector and Generative AI workloads'. The 'Serverless (non-vector)' tab is described as 'A more traditional database solution without any of our new vector capabilities'. Below the tabs, there is a 'Database name' field with the value 'verizon_workshop' and a note: 'Give it a memorable name – this can't be changed later.' The 'Provider' section has three buttons: 'Google Cloud', 'Amazon Web Services' (selected), and 'Microsoft Azure'. The 'Region' section has a dropdown menu with the value 'us-east-2'. At the bottom right, there are 'Cancel' and 'Create database' buttons.

Click **Create Database** and wait for it to show **Active** status (green text)

← Databases

verizon_workshop

Active

ID 85c04a9d-4d74-4138-94d3-76002e38d0f4 • API_ENDPOINT AWS us-east-2 https://85c04a9d-4d74-4138-9...

Overview

Data Explorer

Settings

Or, if you go back out to the list of databases, you will see it here:

Name	Database ID	Reads	Writes	Storage	Data transfer	Vector rea...	Vector writ...	Created	Last used	Status
rbc_workshop	ad4327ed-....	0	0	0.00	0.00	0	0	8/18/2025	8/18/2025	Active

Next, create a database access token

← Databases

verizon_workshop

Active

ID 85c04a9d-4d74-4138-94d3-76002e38d0f4 • API_ENDPOINT AWS us-east-2 https://85c04a9d-4d74-4138-9...

Overview

Data Explorer

Settings

Key Metrics

Requests

Total Latency by Percentile

1.6 ms

1.2 ms

0.8 ms

0.4 ms

0.0 ms

8:51:23 PM UTC | 2025-09-299:00:53 PM UTC | 2025-09-29

p99 Reads

p50 Reads

p99 Writes

p50 Writes

Total Throughput

0 Requests

0.12 RPS

0.09 RPS

0.06 RPS

0.03 RPS

0.00 RPS

8:51:23 PM UTC | 2025-09-299:00:53 PM UTC | 2025-09-29

Total Throughput

Read Throughput

Write Throughput

Average Throughput

Database Details

API Endpoint

us-ea... https://85c04a9d-4d74-...

Application Tokens

Generate Token

Region

Add Region +

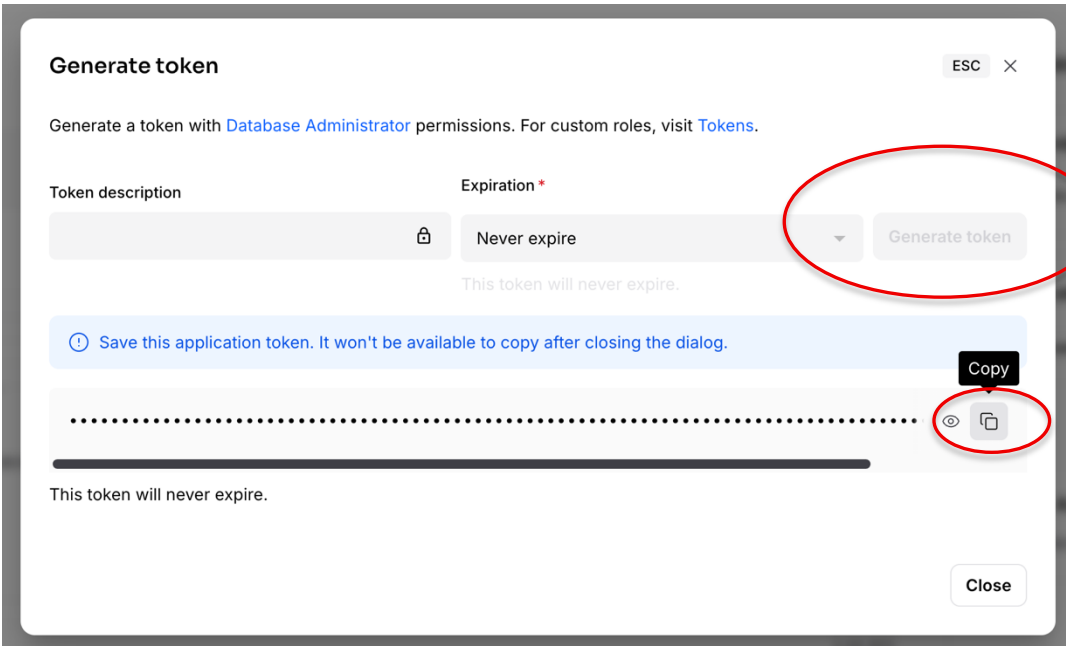
Amazon Web ...

us-e...

Datacenter ID: 85c04a9d-4d74-...

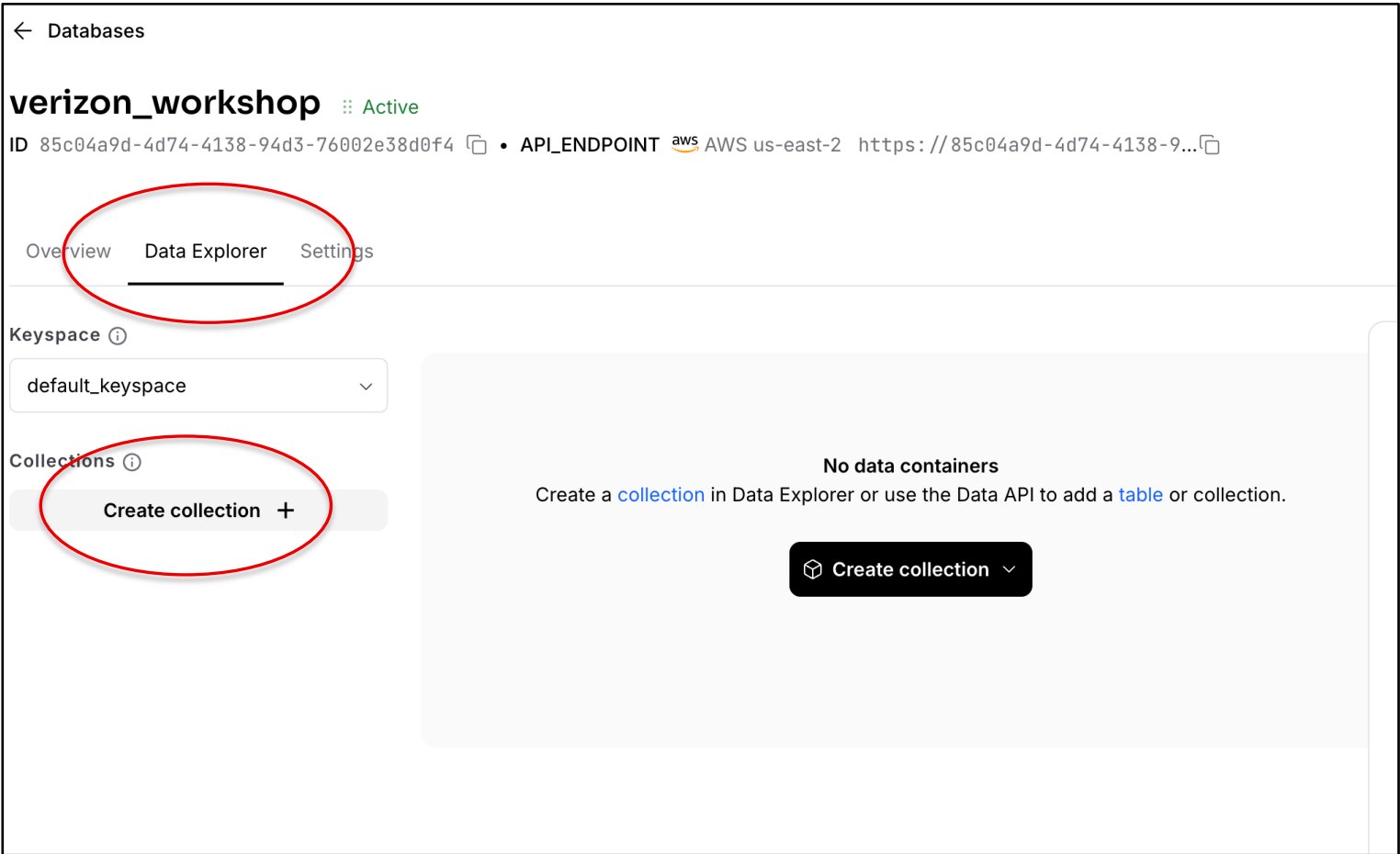
[Back to Top](#)

Click **Generate Token** and then select the Copy icon. Store the token in a plain text file or somewhere generally safe. We'll use it to create a **Global Variable** in just a few steps.



Create a Collection

On your database details page, select **Data Explorer** and then **Create collection**



Configure your collection as shown in the next screenshot below and select **Create Collection**

Create collection ESC ×

Collection name *
vcg_docs

☒ Vector-enabled collection ⓘ

Embedding generation method *
Bring my own

Select a configured [embedding provider integration](#) or generate your own embeddings

Dimensions* ⓘ
384

Value	Example model
1536	text-embedding-ada-002
1024	bge-large-en-v1.5, ember-v1, gte-large...
768	bge-base-en-v1.5, gte-base, instructor-xl...
512	jina-embedding-s-en-v1, distiluse-base-multilingu...
384	bge-small-en-v1.5, gte-small, e5-small-v2...
300	komninos, glove.6B.300d...

Similarity metric* ⓘ
Cosine

Method used to calculate vector similarities

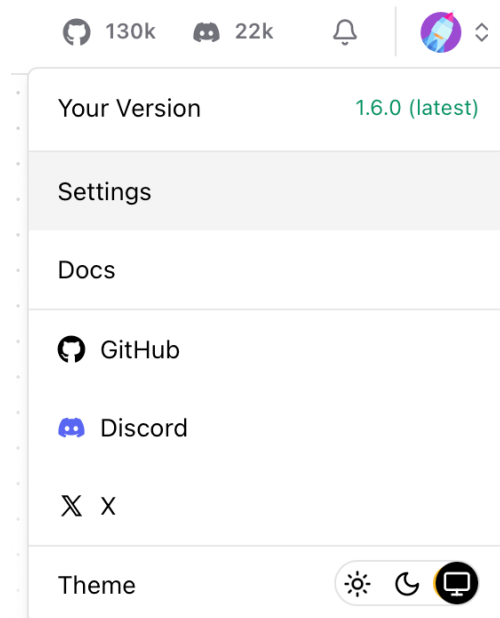
Cancel Create collection

Collection name: `vcg_docs`
Embedding generation method: `Bring my own`
Dimensions: `384`
Similarity metric: `Cosine`

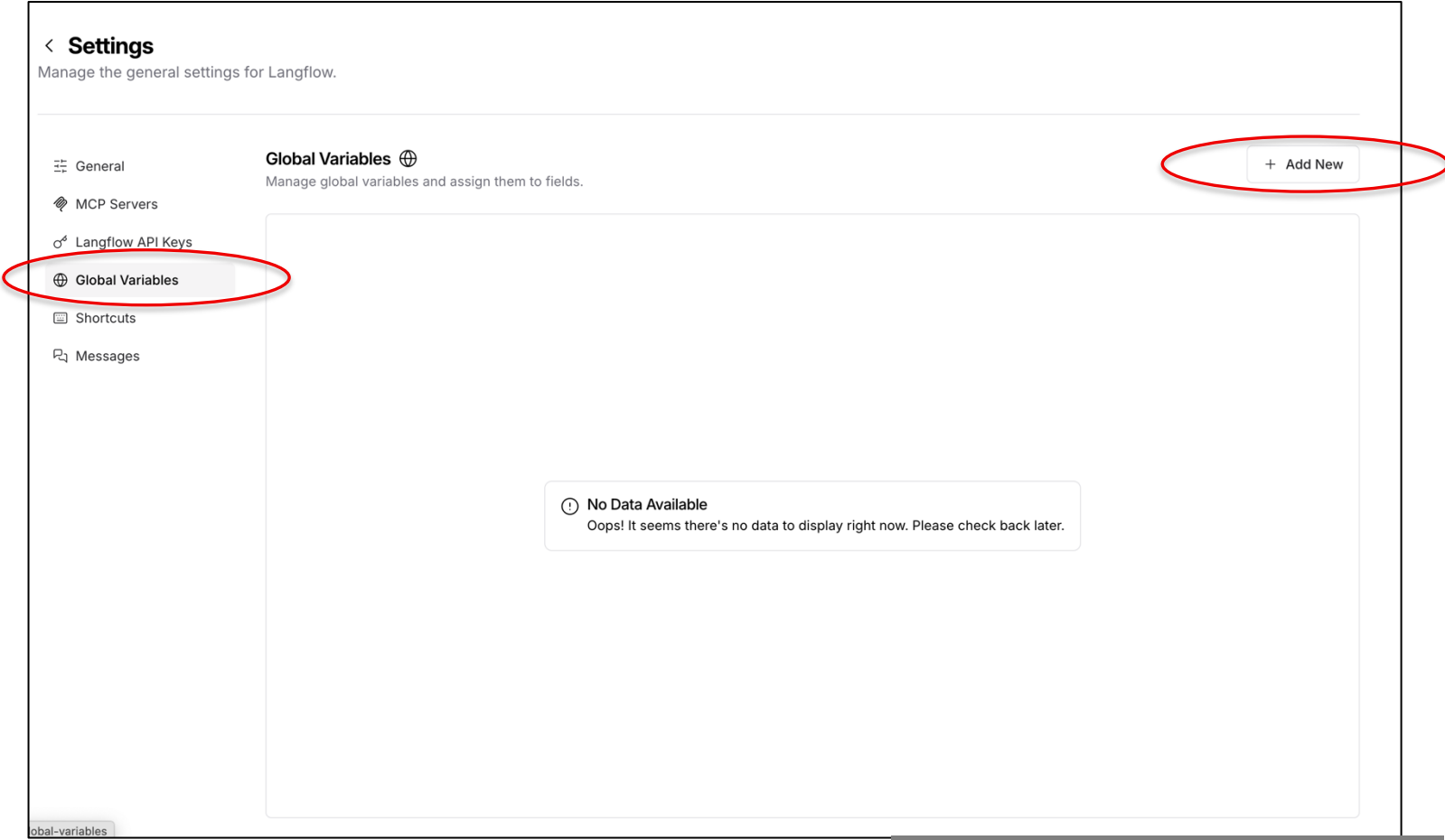
Store Your Token and Keys

Next we will create Global Variables using your Astra and watsonx.ai keys
This will make the rest of the workshop go so much smoother :-)

Start Langflow on your machine and navigate to the UI
In the top right corner of your browser, click the icon, and click **Settings**



Select **Global Variables** from the left-hand navigation, then select **Add New**



Name your first variable **ASTRA_DB**, and **paste in your Astra token** from your plain text editor. Click **Save Variable**.

There is a ZIP file in the GitHub Repo named `watsonx_api_keys.zip`. Download it and open it with the password shared by the presenter.

Paste in the project ID and API key, don't literally type what's below
Add two more Global Variables as shown below:

Name: **WATSONX_PROJECT**
Value: `{project_id}`

Name: **WATSONX_API**
Value: `{api_key}`

You should see something like the below image:

Global Variables ⓘ	
Manage global variables and assign them to fields.	
☐ Variable Name	Type
☐ ASTRA_DB	Credential
☐ WATSONX_PROJECT	Credential
☐ WATSONX_API	Credential

Create Variable

This variable will be available for use across your flows.

Type*

CredentialGeneric

Name*

ASTRA_DB

Value*

your astra token

Apply to fields

Choose a field for the variable...

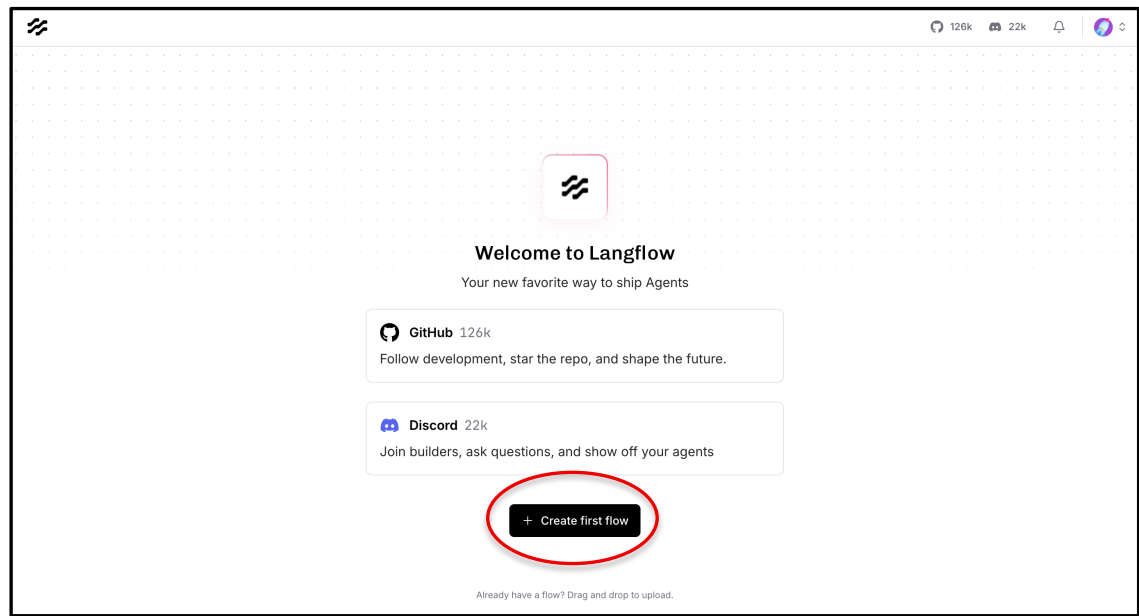
Selected fields will auto-apply the variable as a default value.

Cancel

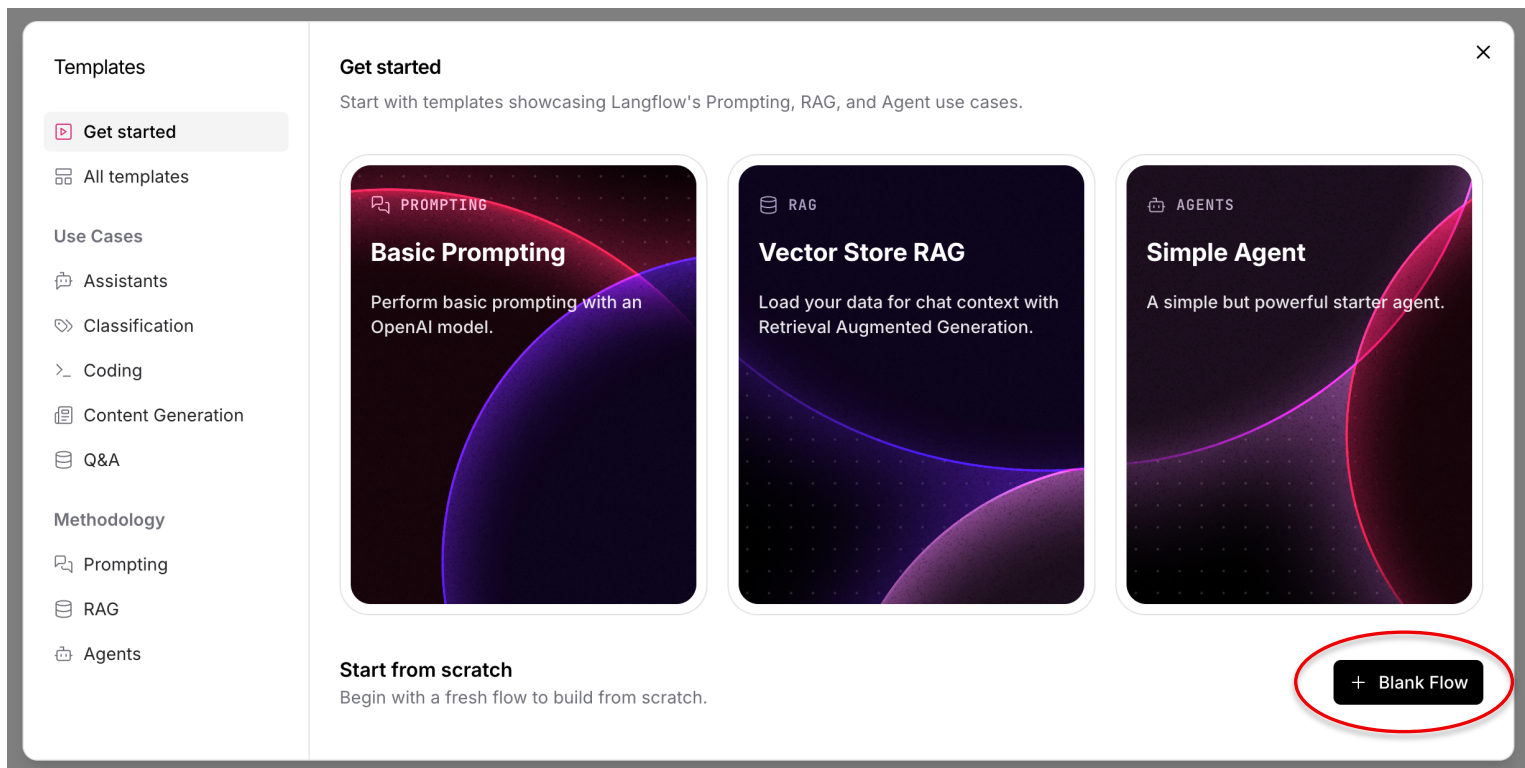
Save Variable

Exercise 1: Hello World Chatbot

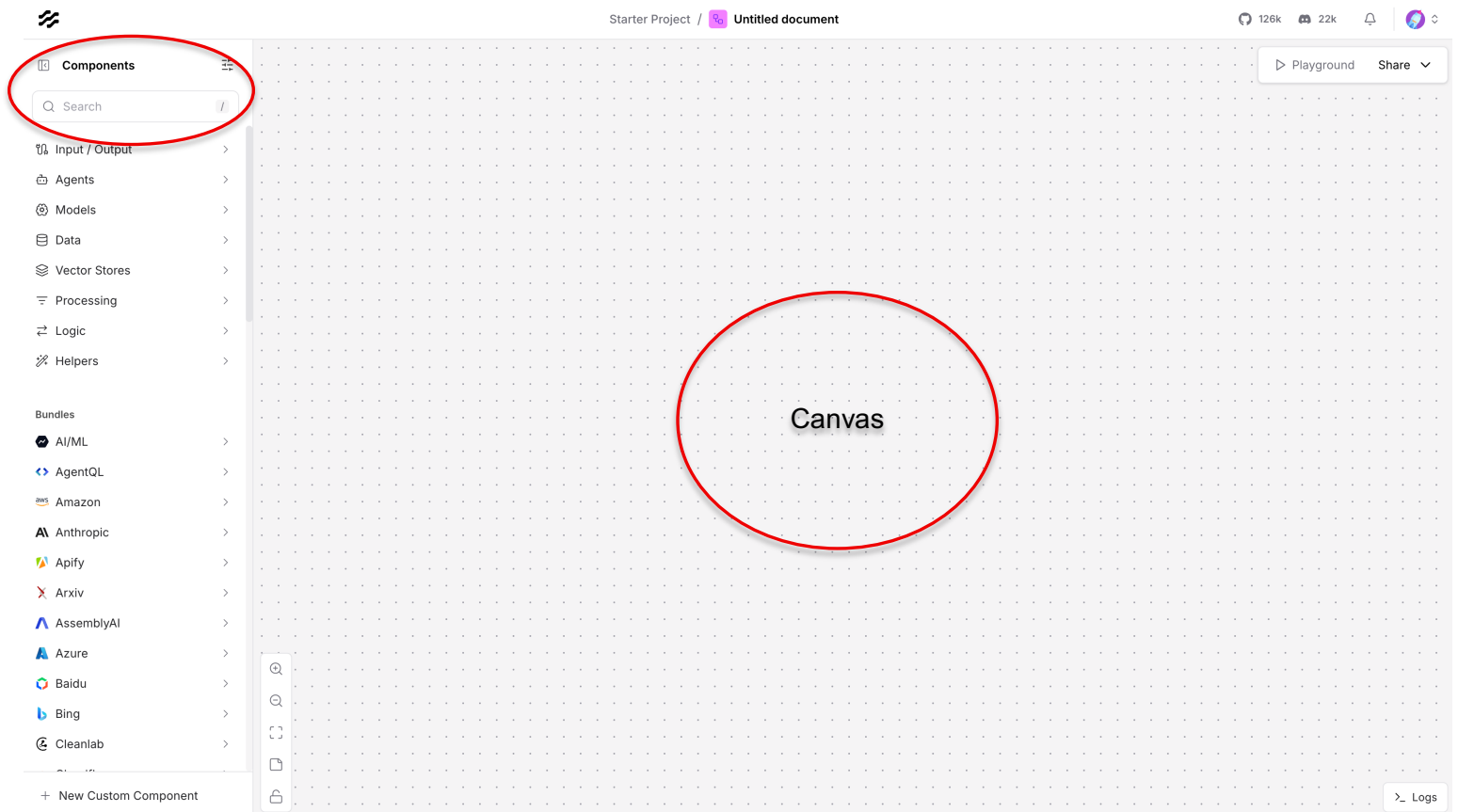
Click the Langflow icon in the upper left of your window. Click **Create first flow** to get started



Click **“Blank Flow”**



You should see a blank “canvas” and the Components toolbar in the left-hand navigation pane



In the Search field under Components, type **IBM**.

Click the plus sign next to **IBM watsonx.ai**

This will add the **IBM watsonx.ai** component onto the canvas

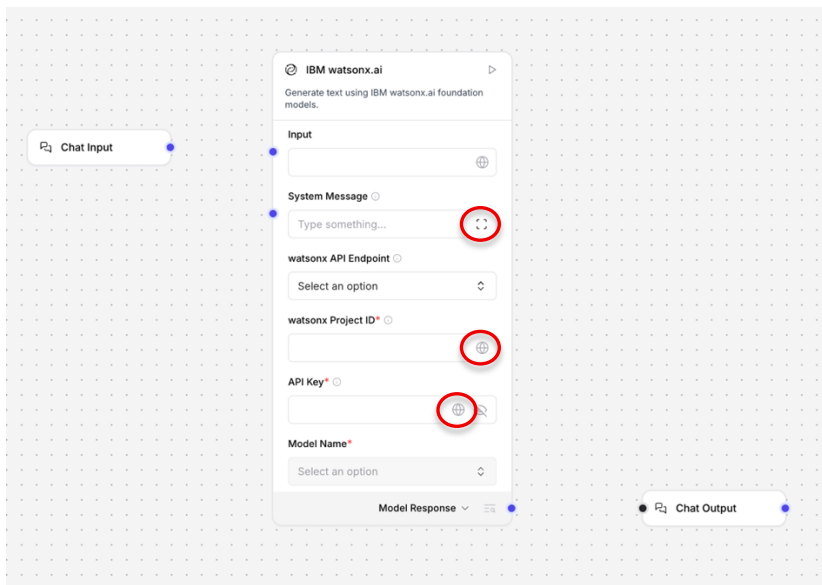
Clear the Component search field by clicking X and type “**Chat**”

Hover over **chat input**, click the plus sign. Same for **chat output**

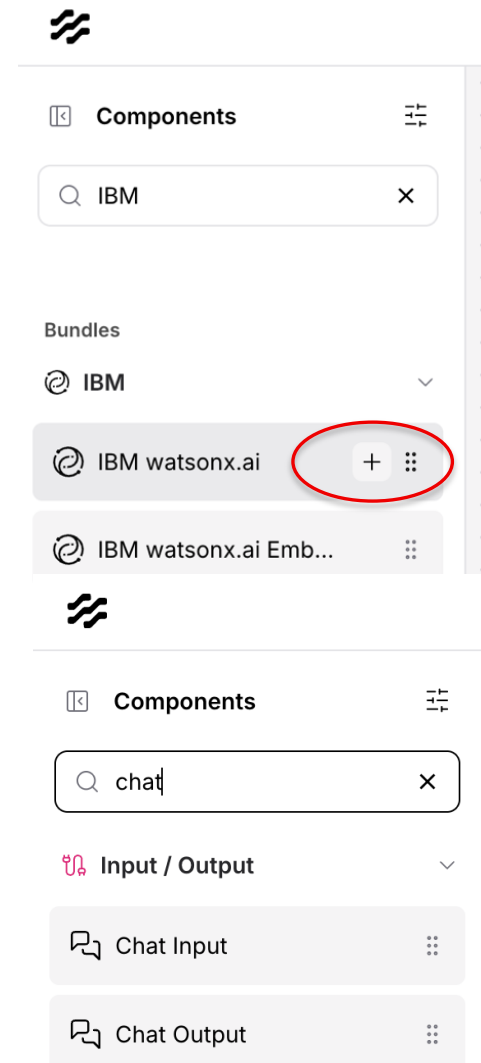
You should now have three components on the canvas

- IBM watsonx.ai
- Chat Input
- Chat Output

You can click on Chat Input and Chat Output and drag them around the canvas with input on the left and output on the right.



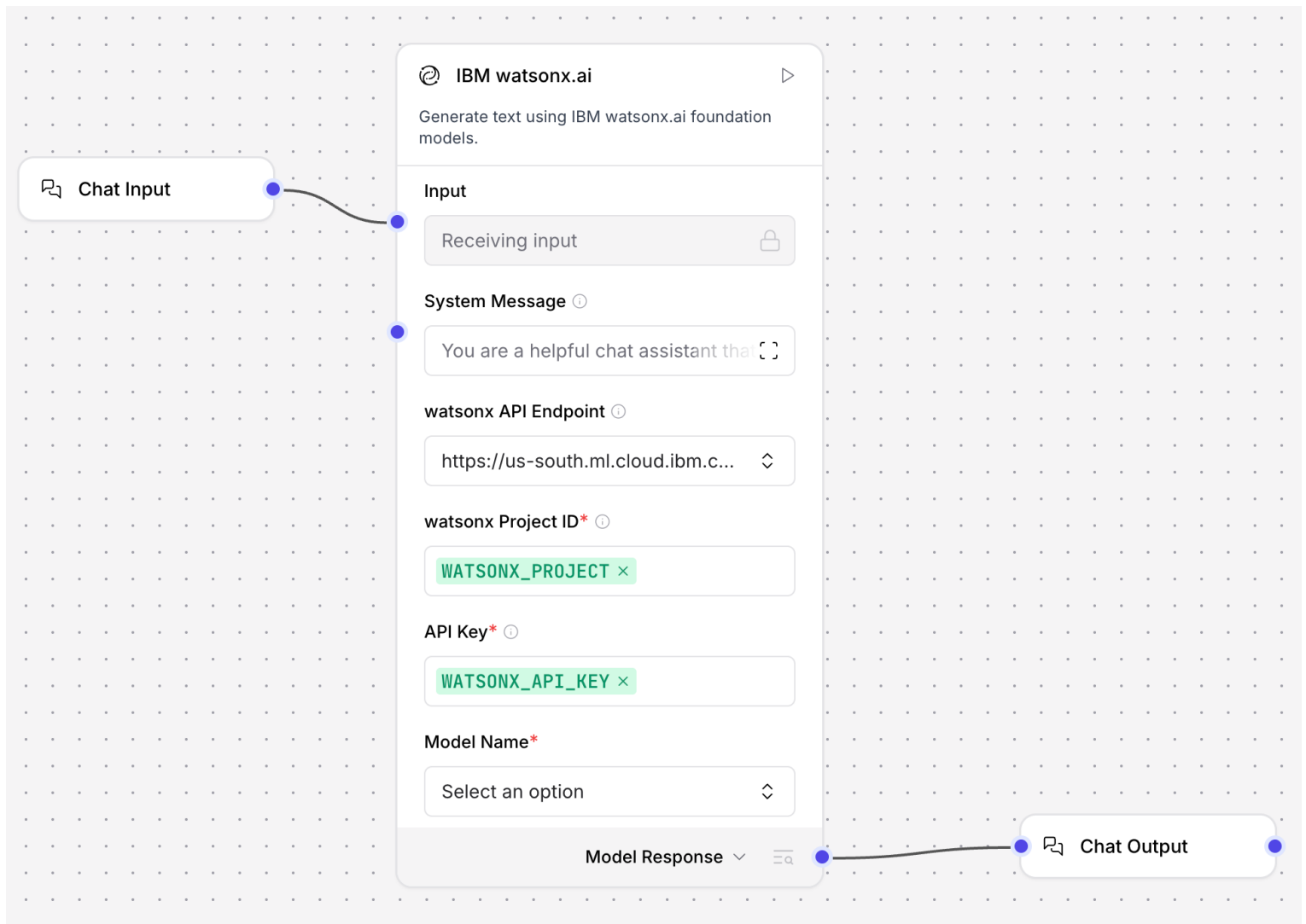
[Back to Top](#)



Connect the Dots

- Click on the **blue** dot on the right side of Chat Input drag the line to the IBM watsonx.ai **Input**
- Do the same for **Model Response** and Chat Output
- Here are the parameters for the watsonx.ai component. Click the square box and globe icons to modify. See above screenshot.
- **Do not literally type** the name of the Global Variable (i.e. WATSONX_API) but click on the globe icon circled in red in the screenshot above and select your variable.
 - System Message
You are a helpful chat assistant that helps users understand technology, and the mysteries of the universe.
 - watsonx API Endpoint: <https://us-south.ml.cloud.ibm.com>
 - watsonx Project ID: **WATSONX_PROJECT**
 - API Key: **WATSONX_API**
 - Model Name: [openai/gpt-oss-120b](#)

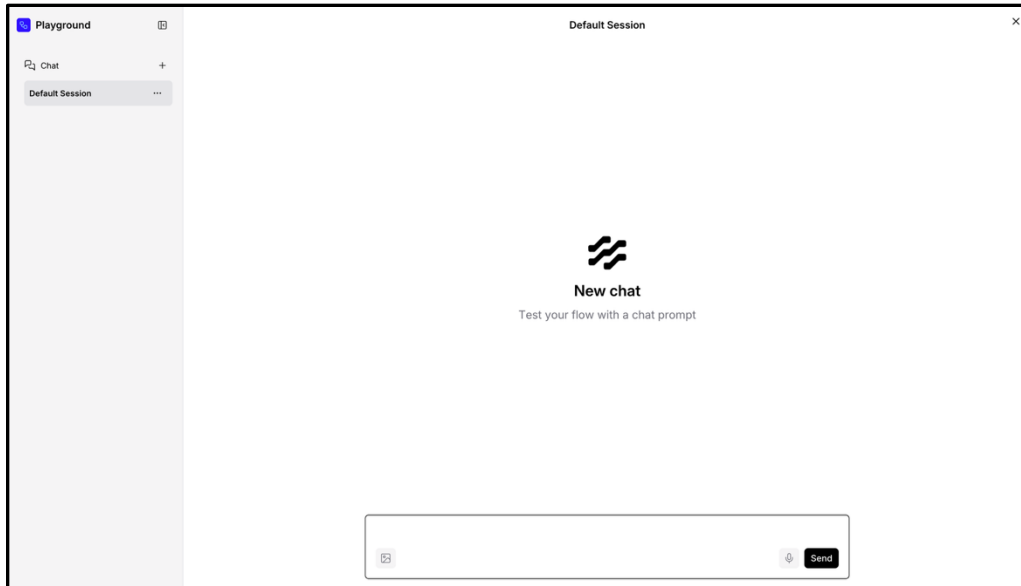
This is the completed Hello World flow:



Playground Share

In the top right corner of the screen, click **Playground**

This will give you a “chatbot” style interface (and ensure we have good connectivity to watsonx.ai)



Ask a question such as “**what is today’s date? And who are you?**” to ensure you have connectivity to IBM Cloud and the watsonx.ai large language models hosted there.

Default Session



User

what is today's date? and who are you?



AI openai/gpt-oss-120b

Today's date is **September 30, 2025**.

I'm ChatGPT, an AI language model created by OpenAI. I'm here to help answer your questions, explain technology, explore scientific concepts, and discuss a wide range of topics. Feel free to ask me anything!

Close the Playground by clicking the X in the top right

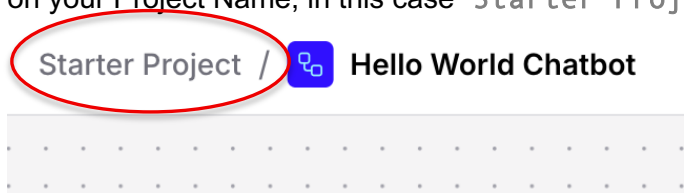
Rename your flow by hovering over **New Flow** above your watsonx.ai component

Click the Pencil icon, and rename your flow **Hello World Chatbot**. Click **Save** to continue

The screenshot displays the Langflow application interface. At the top, the breadcrumb navigation shows 'Starter Project / New Flow', with 'New Flow' highlighted by a red circle and a pencil icon. The main workspace on the left contains a flow diagram with a '2ms' label and a connection to an 'IBM watsonx.ai' component. The component's configuration panel on the right includes an 'Input' field with 'Receiving input', a 'System Message' field with 'You are a helpful chat assistant that', and a 'watsonx API Endpoint' field with 'https://us-south.ml.cloud.ibm.c...'. On the right side, a modal dialog titled 'New Flow' is open. It has a 'Name' field containing 'Hello World Chatbot', a 'Description' field with 'Empowering Enterprises with Intelligent Interactions.', and a 'Lock Flow' toggle switch. At the bottom of the dialog are 'Cancel' and 'Save' buttons.

Exercise 2: Document Ingestion with Metadata Enrichment

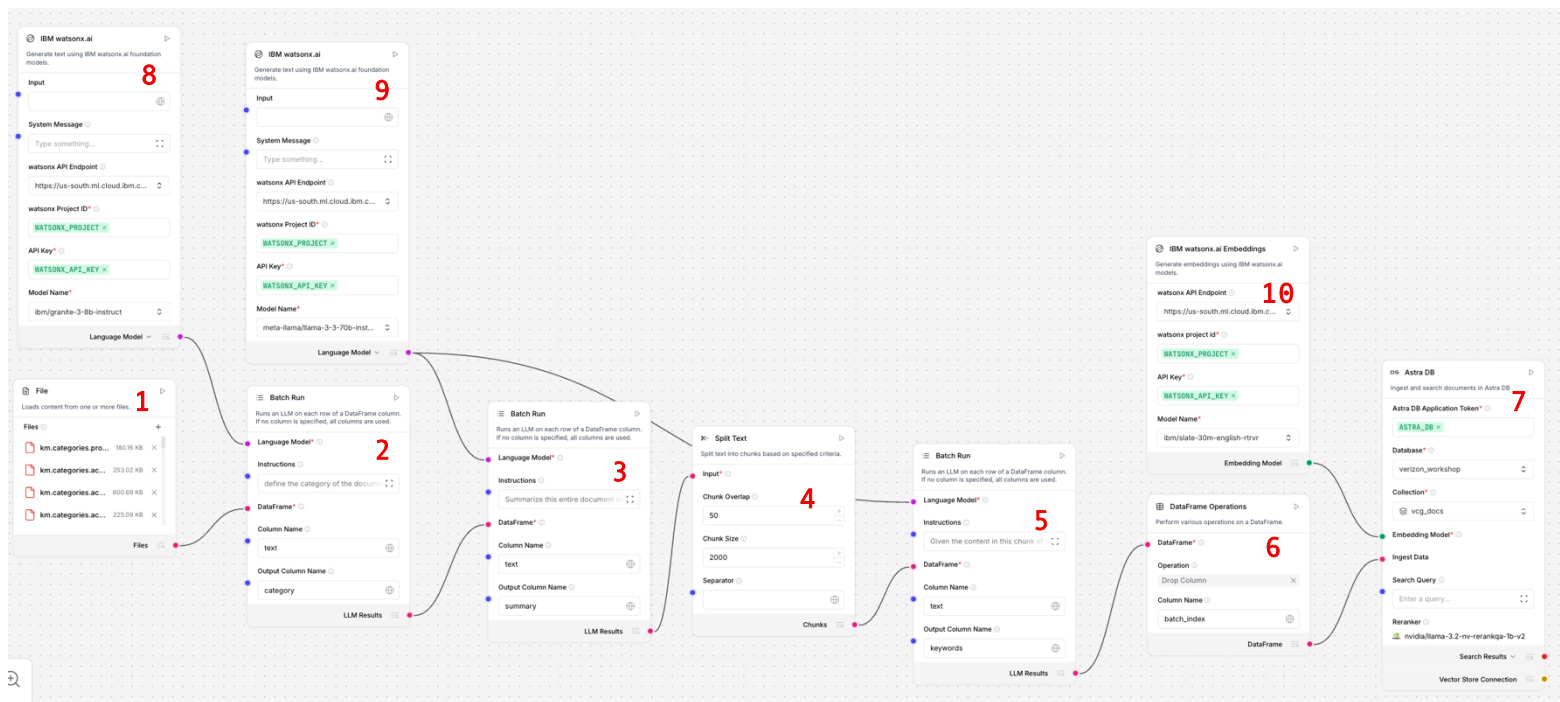
You can always get back to the main Langflow screen by clicking on the Langflow icon in the top left, or by clicking on your Project Name, in this case “Starter Project.”



Click new flow, and blank flow as we did in the last exercise. Name your new flow **Enriched RAG Ingest**.

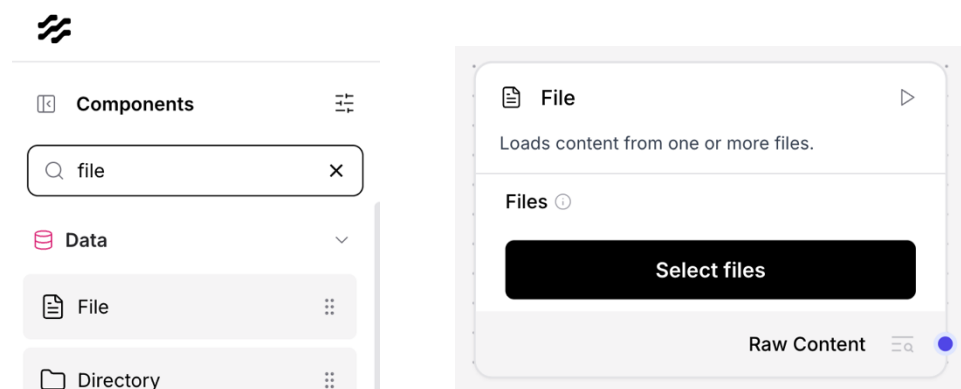
Enhanced RAG Ingest Overview

This is the enriched ingest flow that we will be creating. You can zoom in on this image to see its various components. Let’s go through it step by step.

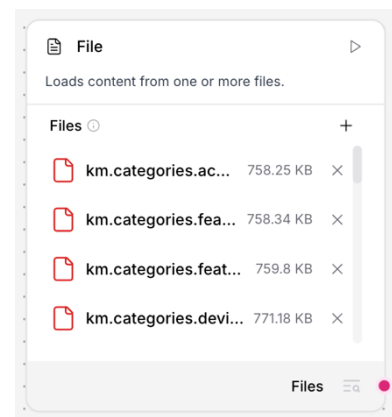
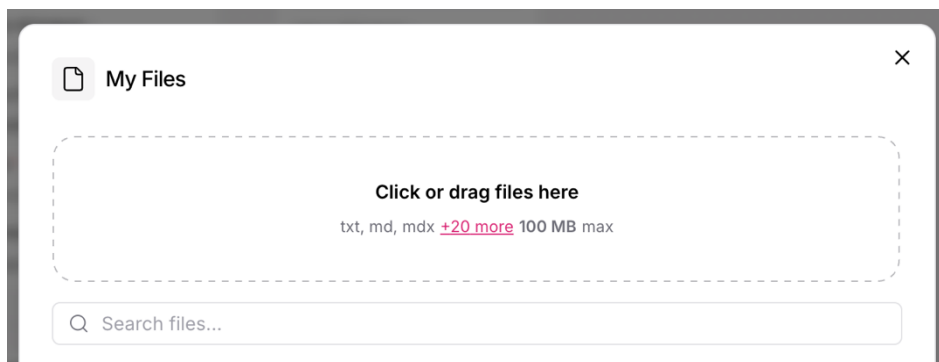


1: File Component

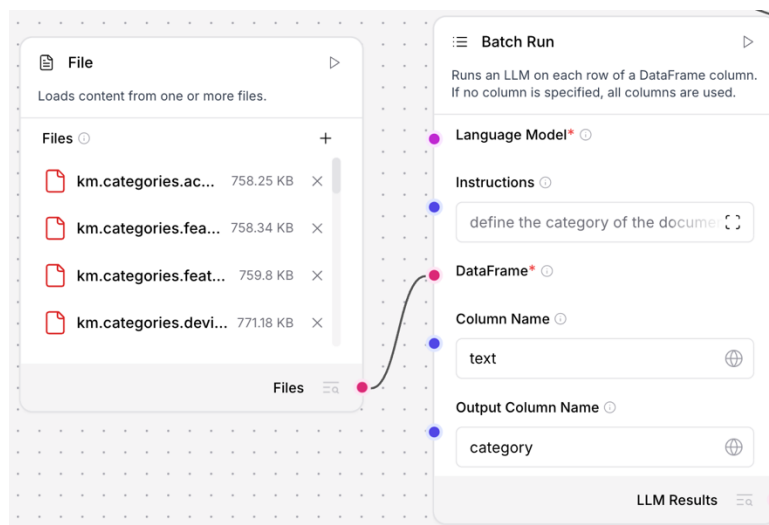
The **File** component will accept PDF (or just about any type) files and load them into the flow
Use the search field in your left-hand navigation components pane to find the **File** component
Click the plus sign to drop it onto the canvas, alternatively you can click the **File** card and drag it onto the canvas



Download the ZIP file located in the GitHub repo with the rest of the workshop material.
Locate the zip file and unzip it. It should be called **selected_pdfs.zip**. You should see ~30 PDF files.
Click **Select Files** from your **File** component in Langflow, find the directory of PDFs that you unzipped
Select all the files (not the directory itself) and drag them into the checkered box to upload them to Langflow



Click “**Select files.**” You’ll now see the list of files and its output: **Files**
You’ll connect the output to the **DataFrame** input of the next component

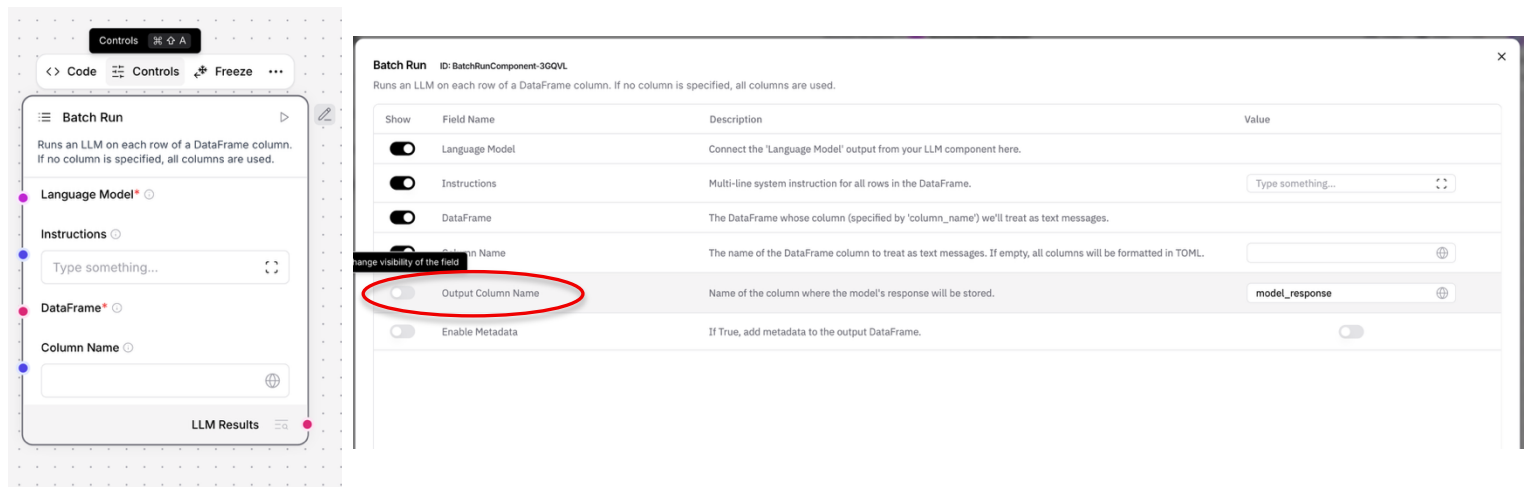


[Back to Top](#)

2: Batch Run

Batch Run will take each row of the **DataFrame** and use an LLM to perform an action, resulting in a new column within the **DataFrame**. Use the Search field to find Batch Run and bring it onto the canvas.

NOTE: sometimes parameters of a component are hidden. Click on an individual component to expose the Controls button at the top of the component. For Batch Run, click the toggle next to Output Column Name



Fill in the Batch Run parameters as follows:

Instructions:

define the category of the document. the category can always be found in the footer of the document as a part of the URL. the format of the URL you are looking for is as follows:

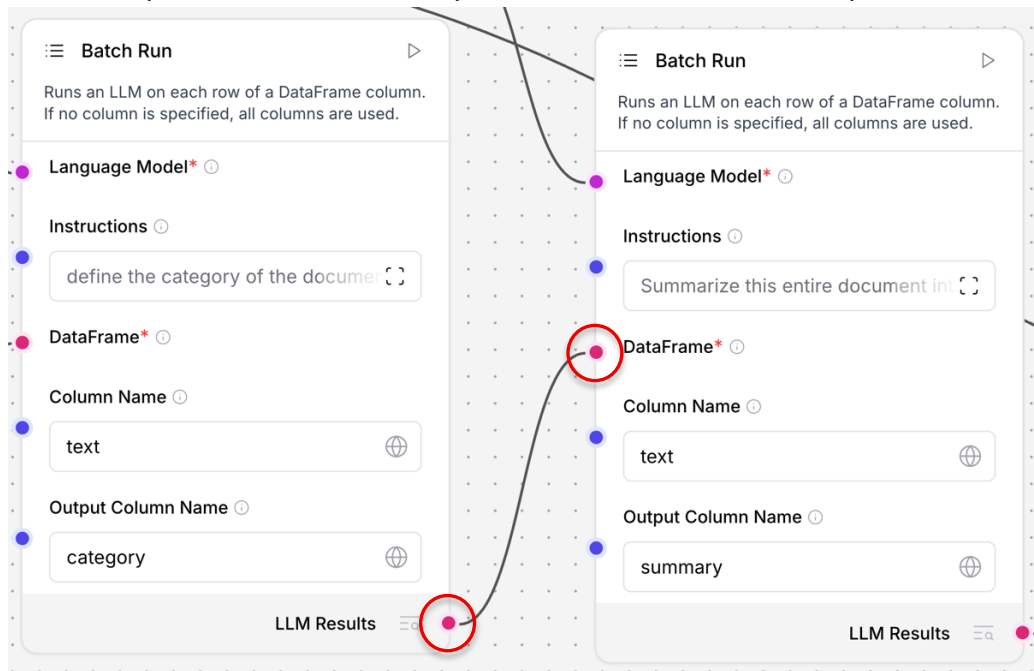
```
file:///Users/michael.obrien/git_repos/verizon/vcg_data/html/vcg-top1k/km.categories.{document_category}.{document_title}.html
```

let your output be only the category found in the {document_category} position within the URL and no other explanation or text.

Column Name: text

Output Column Name: category

Connect the LLM Results output to the DataFrame input of the next Batch Run component



[Back to Top](#)

3: Batch Run

Add another Batch Run component. **TIP:** you can copy and paste components! Fill in the Batch Run parameters as follows:

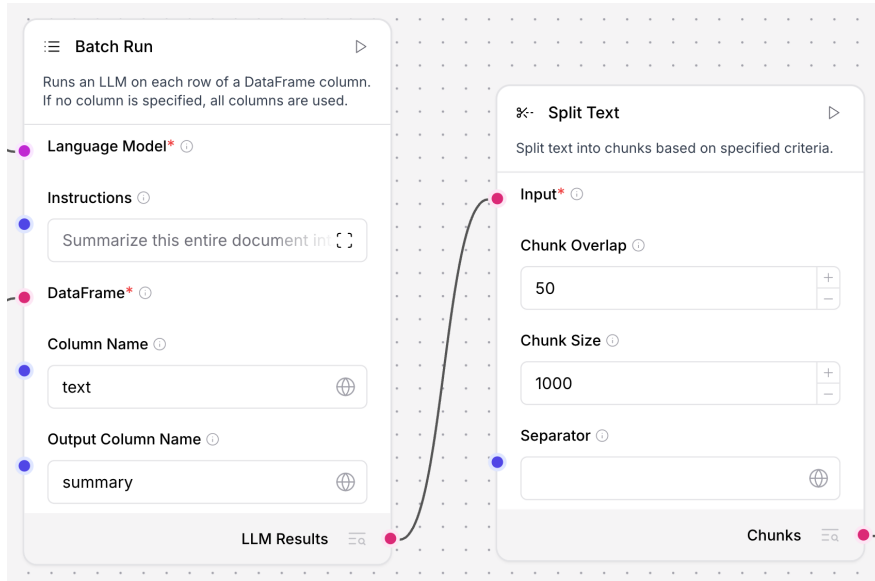
Instructions:

Summarize this entire document into three sentences. Keep your response to strictly three sentences and no more. Do not exceed three sentences. Don't ever say anything like "Here is the summary of the document." Only provide the summary and no other explanation.

Column Name: text

Output Column Name: summary

Connect the LLM Results output to the “Input” of the Split Text component



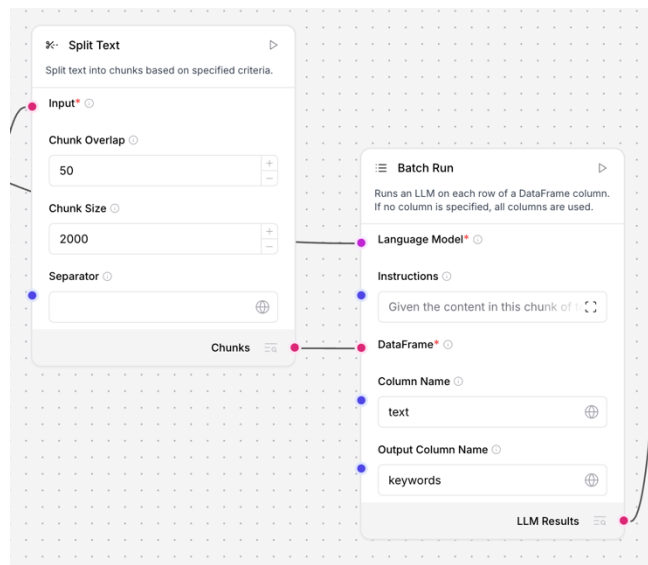
4: Split Text

This component divides the documents into “chunks” to be stored in a vector database. Fill in the parameters as follows:

Chunk Overlap: 50

Chunk Size: 2000

Connect the **Chunks** output of Split Text to the **DataFrame** input of the next Batch Run



5: Batch Run

Add *another* Batch Run component. Fill in the Batch Run parameters as follows:

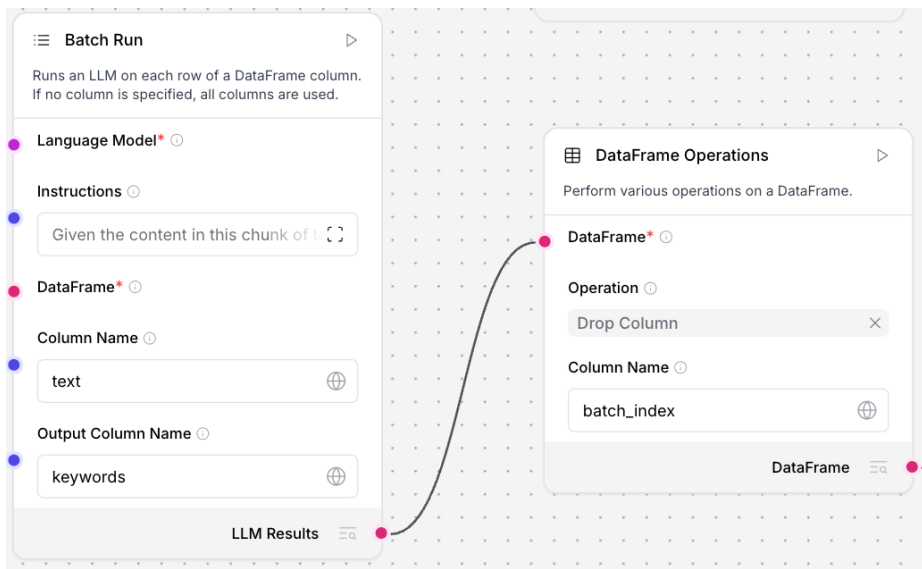
Instructions:

Given the content in this chunk of text, select three to four keywords and return them, each word separated by a comma. Let each keyword be a single word and not a phrase. Do not use multiple words for one single keyword. Only create a list of keywords. Ignore any filler words like "the", "and", "or", and so on. Let your output only be the keywords, each separated by a comma. Do not provide any other explanation or additional text. Do not surround the key words in quotes, brackets, or any other type of formatting.

Column Name: text

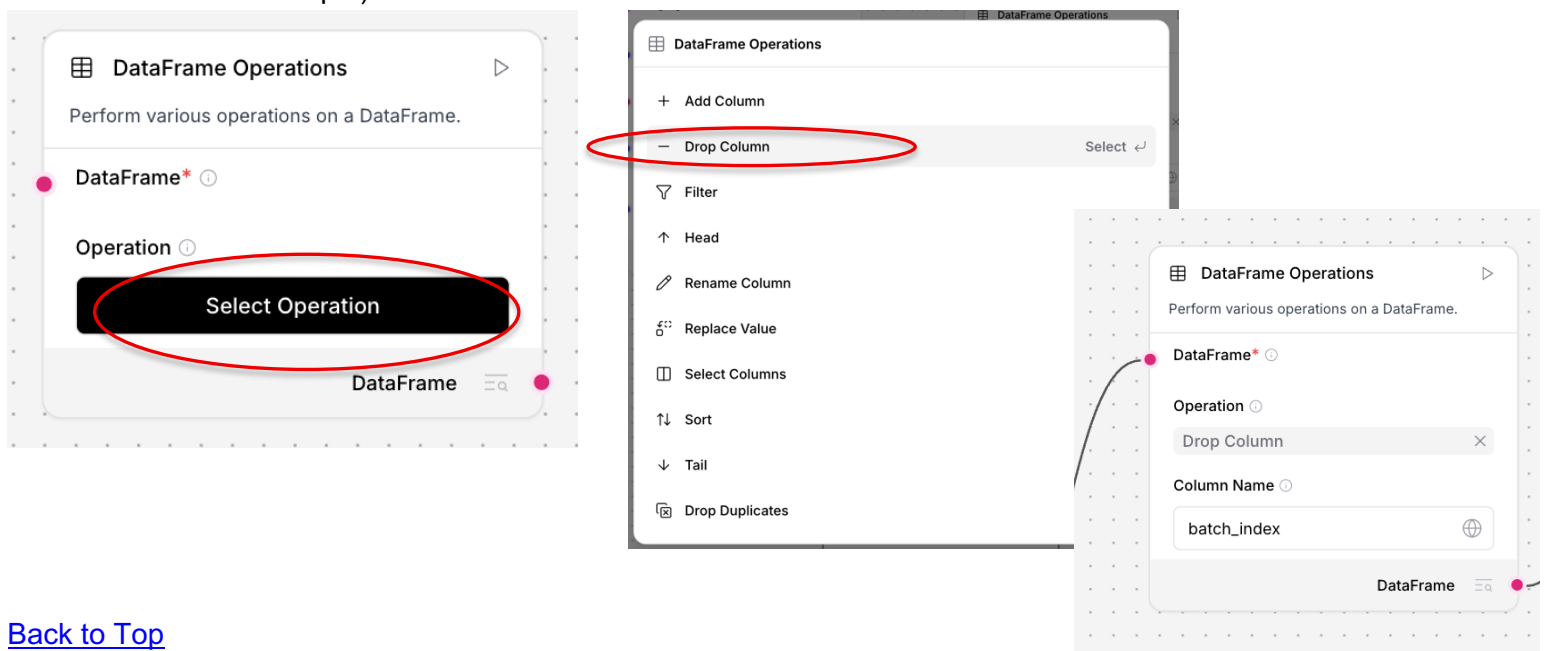
Output Column Name: keywords

Connect the **LLM Results** output to the **DataFrame** Input of the DataFrame Operations component.



6: DataFrame Operations

This component will drop an erroneous column in the **DataFrame**. Click Select Operation, then select Drop Column. In the **Column Name** parameter, enter `batch_index`. Wire LLM Results from the previous Batch Run component into the DataFrame input. Wire the DataFrame output into the Ingest Data input of Astra DB (see component 7 below for the connection example).



[Back to Top](#)

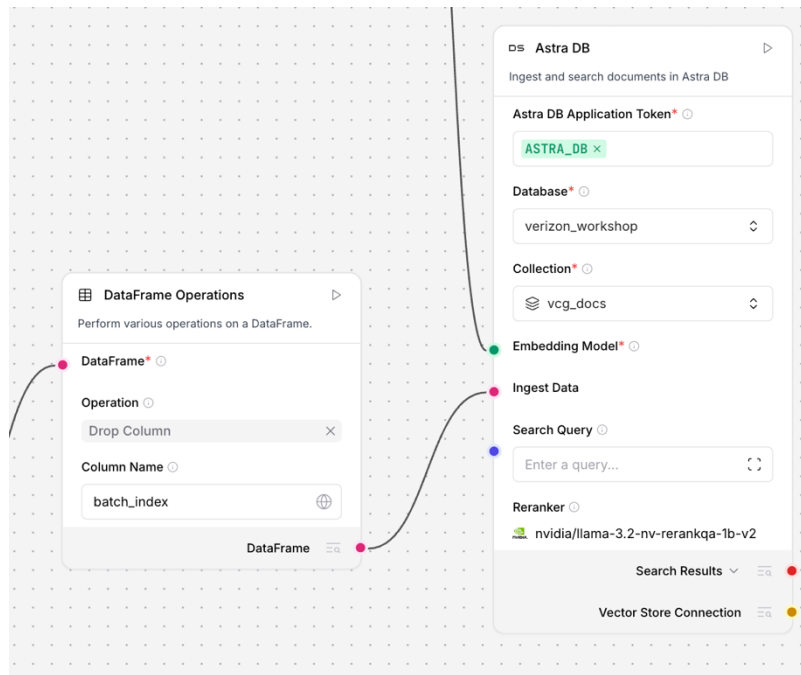
7: Astra DB

Add the Astra DB component. The vector database for our exercise is the database you created for pre-work. Fill in the parameters as follows. The database and collection parameters will be hidden until you select your application token by clicking the globe icon.

Astra DB Application Token: **ASTRA_DB**

Database: **verizon_workshop**

Collection: **vcg_docs**



8: IBM watsonx.ai

Back above component **1** (File), add IBM watsonx.ai. This will serve as the Language Model input for Batch Run component **2**. When you first drag the IBM watsonx.ai component onto the canvas, its output will be Model Response. We need to change it to Language Model. Click on the “down carat” icon and switch the output type to Language Model. See below, and fill in the parameters as follows:

watsonx API Endpoint: <https://us-south.ml.cloud.ibm.com>

watsonx Project ID: **WATSONX_PROJECT**

API Key: **WATSONX_API**

Model Name: **ibm/granite-3-3-8b-instruct**

9: IBM watsonx.ai

Add a second instance of watsonx.ai, change its output to Language Model, and connect it as shown in the screen shot below to Batch Run **3** and **5**.

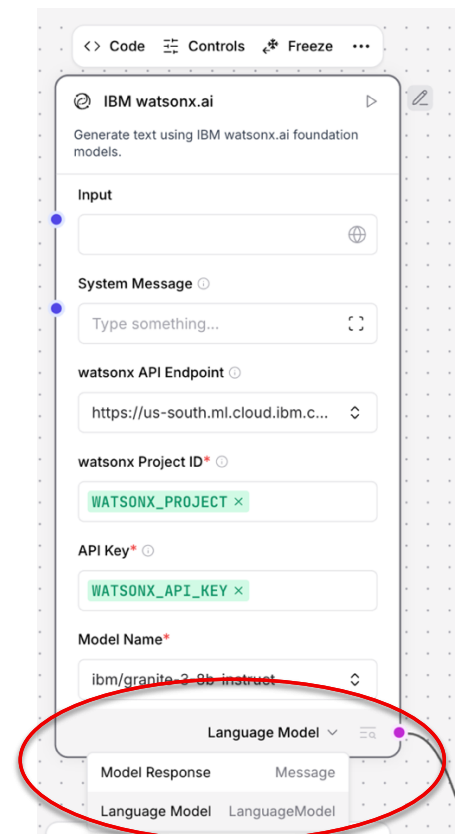
Fill in the parameters as follows:

watsonx API Endpoint: <https://us-south.ml.cloud.ibm.com>

watsonx Project ID: **WATSONX_PROJECT**

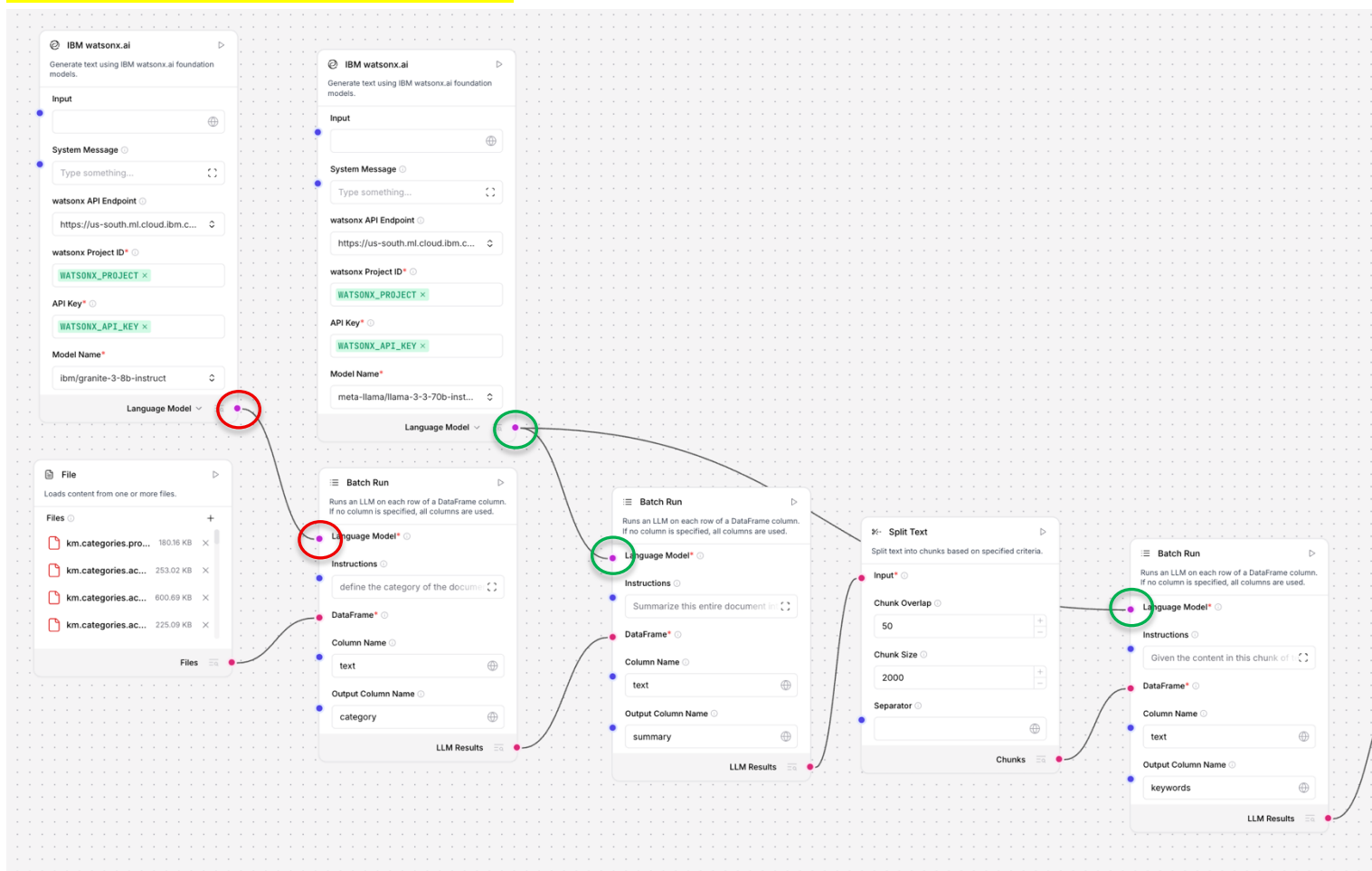
API Key: **WATSONX_API**

Model Name: **meta-llama/llama-3-3-70b-instruct**



[Back to Top](#)

See the below screen shot for connections:



10: IBM watsonx.ai Embeddings

Finally, add the IBM watsonx.ai Embeddings component next to the Astra DB component, number 7. Fill in the parameters as follows:

watsonx API Endpoint: <https://us-south.ml.cloud.ibm.com>

watsonx Project ID: **WATSONX_PROJECT**

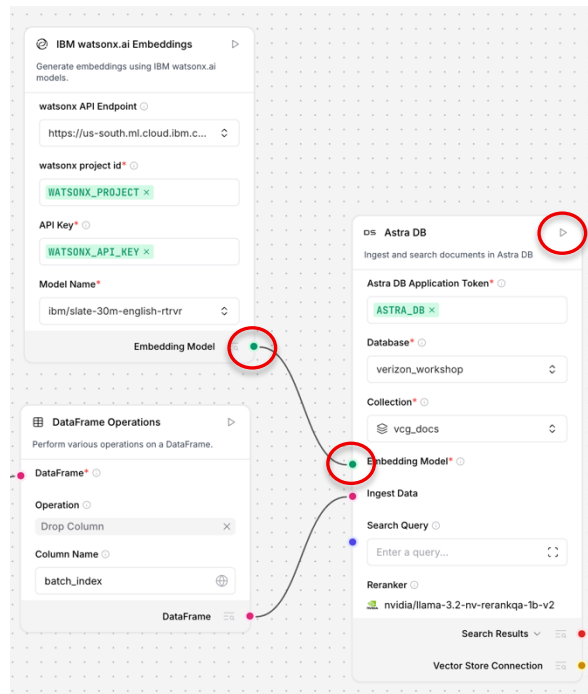
API Key: **WATSONX_API**

Model Name: **ibm/slate-30m-english-rtrvr-v2**

Connect the Embeddings output of the IBM watsonx.ai Embeddings component to the Embeddings input of the Astra DB component as shown

Run Your Flow!

Click the Play icon at the top right of the Astra DB component to run your document ingestion flow.

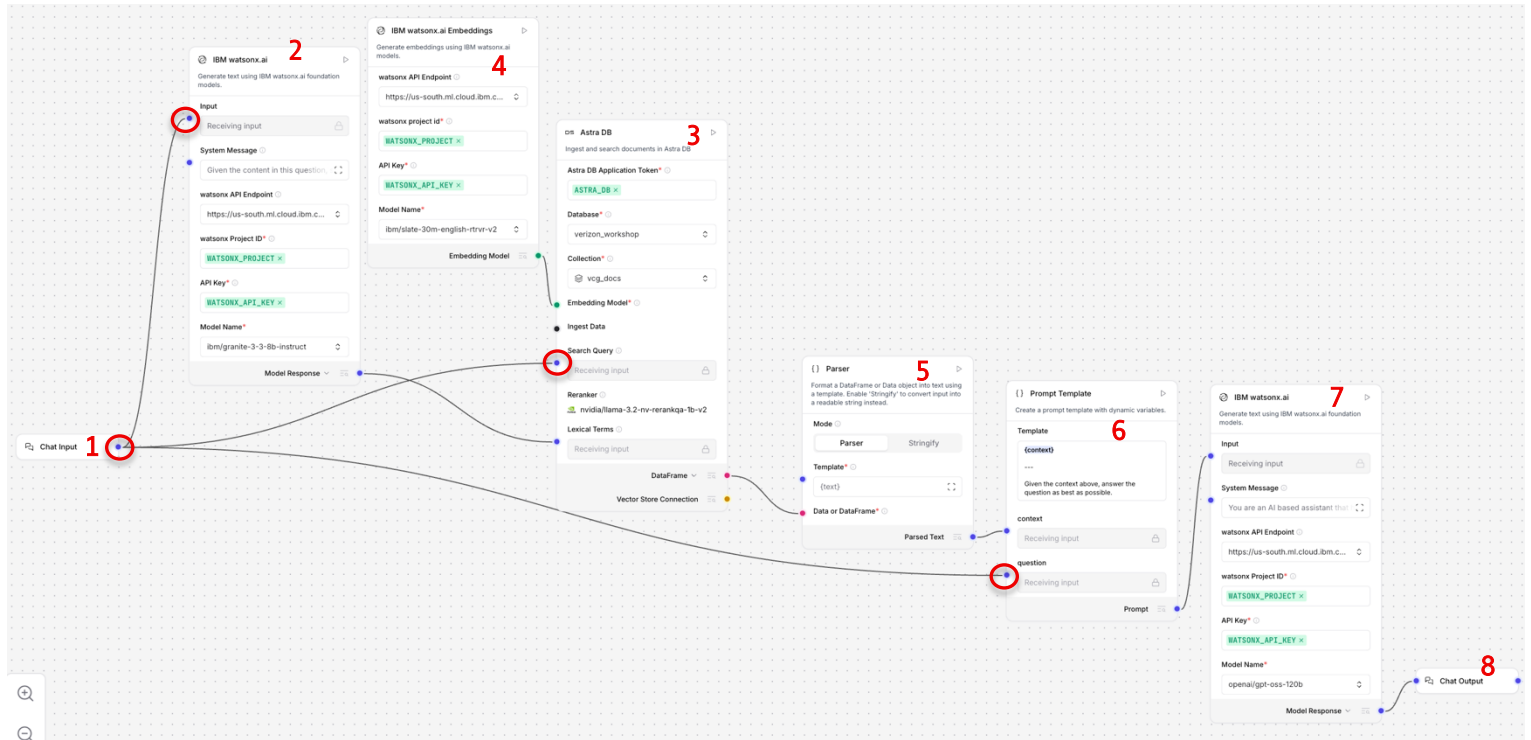


[Back to Top](#)

Exercise 3: Enriched Data Retriever Flow

Retriever Flow Overview

This is the retriever flow that we will be creating. You can zoom in on this image to see its various components. Let's go through it step by step.



Navigate back to your Starter Project screen, click New Flow, Blank Flow, and name your new flow Enriched RAG Retriever.

1: Chat Input

Just like in the “Hello World” exercise, start with the Chat Input, but we’ll be making three connections. See red circles above. Refer to this overview as components are added if you need to.

2: IBM watsonx.ai

This instance of wx.ai needs to keep the **Model Response** output setting. Connect Chat Input to Input. Fill out the rest of the parameters as follows:

System Message:

Given the content in this question, select three to four keywords and return them without any further explanation or formatting. Let the keywords be a single word and not a phrase. Do not use multiple words for one single keyword. Only create a list of keywords. Ignore any filler words like "the", "and", "or", and so on.

watsonx API Endpoint: <https://us-south.ml.cloud.ibm.com>

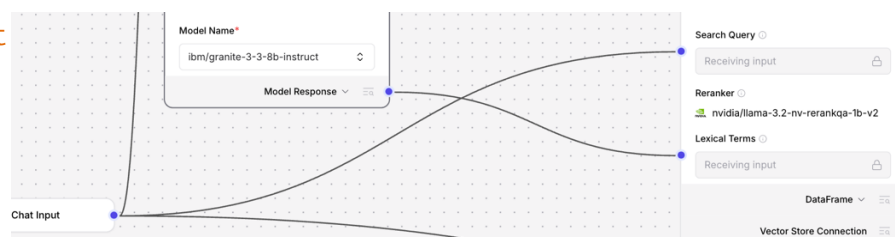
watsonx Project ID: **WATSONX_PROJECT**

API Key: **WATSONX_API**

Model Name: **ibm/granite-3-3-8b-instruct**

Connect the **Model Response** output to the **Lexical Terms** input of the next component,

[Back to Top](#)



3: Astra DB

The vector database for our exercise is the database you created for pre-work.

Fill in the parameters as follows:

Astra DB Application Token: **ASTRA_DB**

Database: **verizon_workshop**

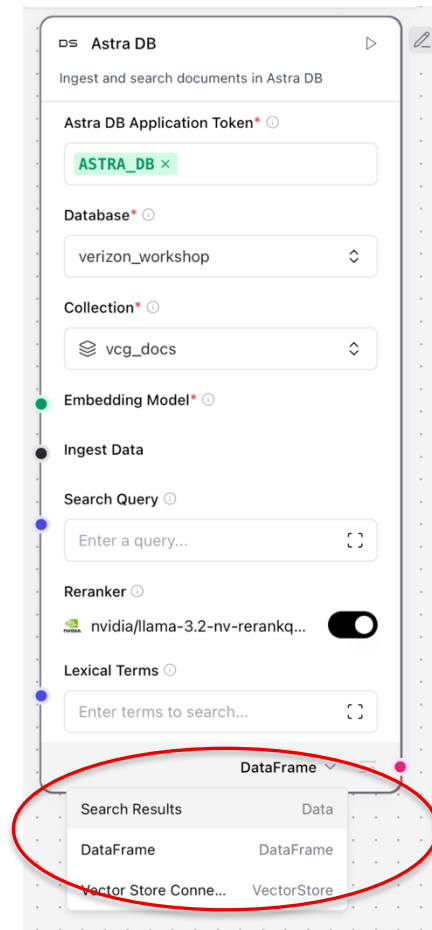
Collection: **vcg_docs**

When the Astra DB component is initially dropped onto the Canvas, the default output is Search Results.

Click the down carat and select DataFrame.

Fill in all of the parameters above, then the **Lexical Terms** input will appear

Connect the Embeddings input to the next component, IBM watsonx.ai Embeddings. Connect the DataFrame output to component **5**, Parser.



4: IBM watsonx.ai Embeddings

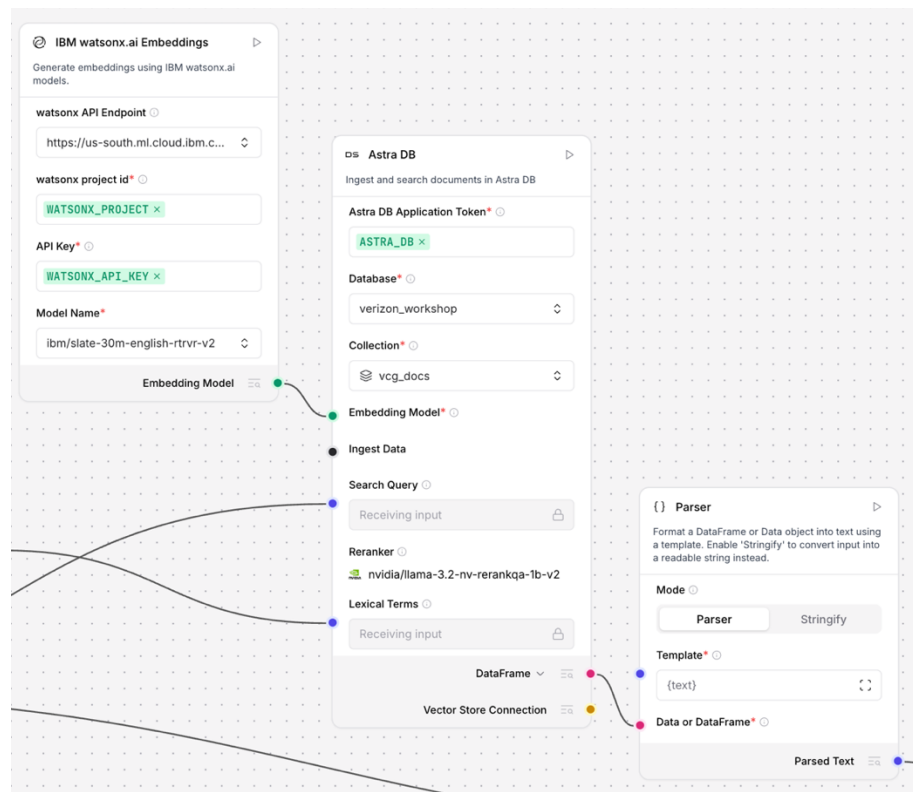
Finally, add the IBM watsonx.ai Embeddings component next to the Astra DB component, number **3**. See the above screenshot for connections.

watsonx API Endpoint: **https://us-south.ml.cloud.ibm.com**

watsonx Project ID: **WATSONX_PROJECT**

API Key: **WATSONX_API**

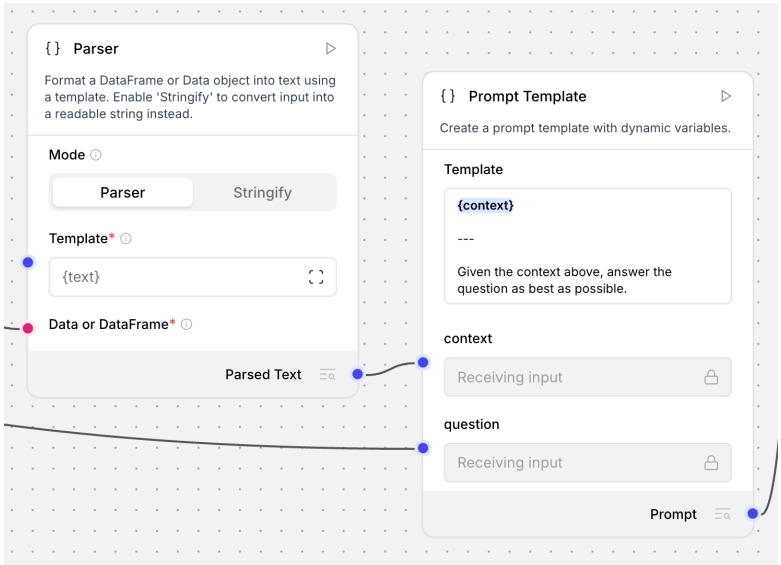
Model Name: **ibm/slate-30m-english-rtrvr-v2**



5: Parser

This component parses out a specific column from a **DataFrame** and passes it to its output as a **Message** (plain text). Connect its **Parsed Text** output to the **Context** input of the next component, Prompt Template.

Notice the Prompt Template also has an input connection on Question from the Chat Input component.



6: Prompt Template

The prompt template component is useful for prompt engineering, as it allows for dynamic inputs to be created. As you write your prompt, any word in **{curly_braces}** automatically becomes an input into the component. Fill out the Template for the prompt like this:

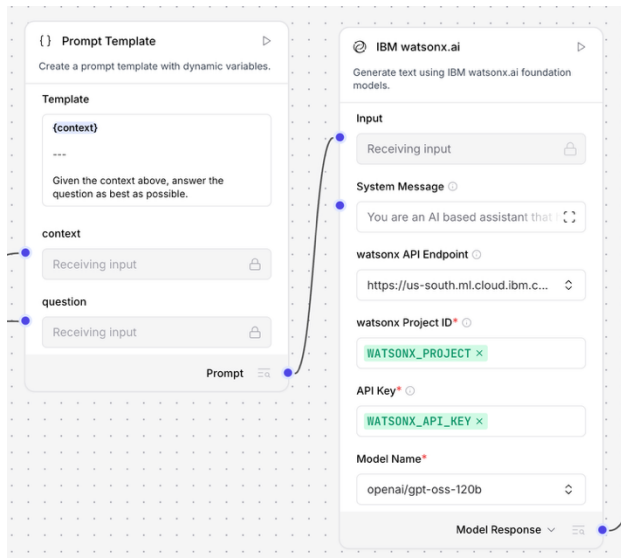
{context}

Given the context above, answer the question as best as possible.

Question: {question}

Answer:

Connect the Prompt output to the Input of the next watsonx.ai component.



7: IBM watsonx.ai

This instance of wx.ai also needs to keep the **Model Response** output setting. Connect the Prompt component's **Prompt** output to **Input**. Fill out the rest of the parameters as follows:

System Message:

You are an AI based assistant that helps internal support users at Verizon find information about their products and services so they, in turn, can help Verizon's customers. Given your input prompt with context, answer the question to the best of your ability.

watsonx API Endpoint: <https://us-south.ml.cloud.ibm.com>

watsonx Project ID: **WATSONX_PROJECT**

API Key: **WATSONX_API**

Model Name: **openai/gpt-oss-120b**

Connect the **Model Response** output to the **input** of Chat Output

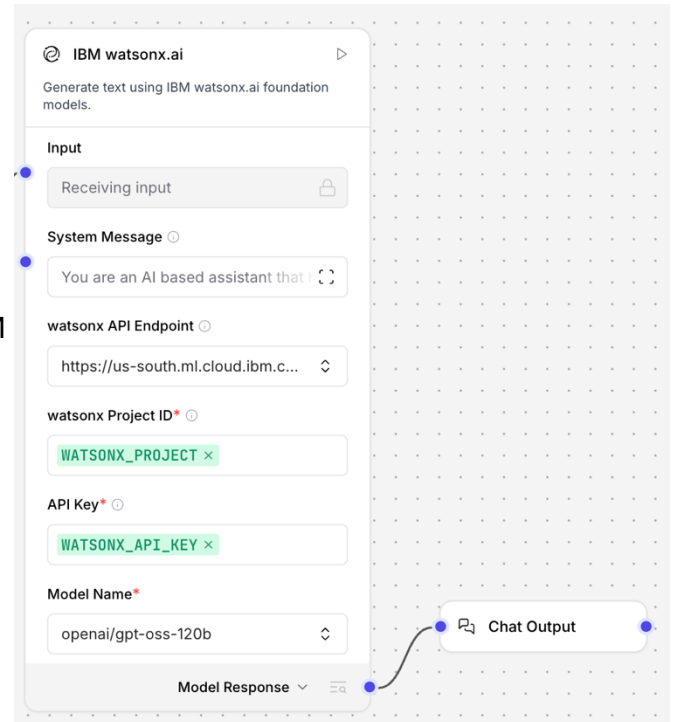
8: Chat Output

The end of the retriever flow is Chat Output, which returns the LLM Model Response back to the user session.

Ask Some Questions

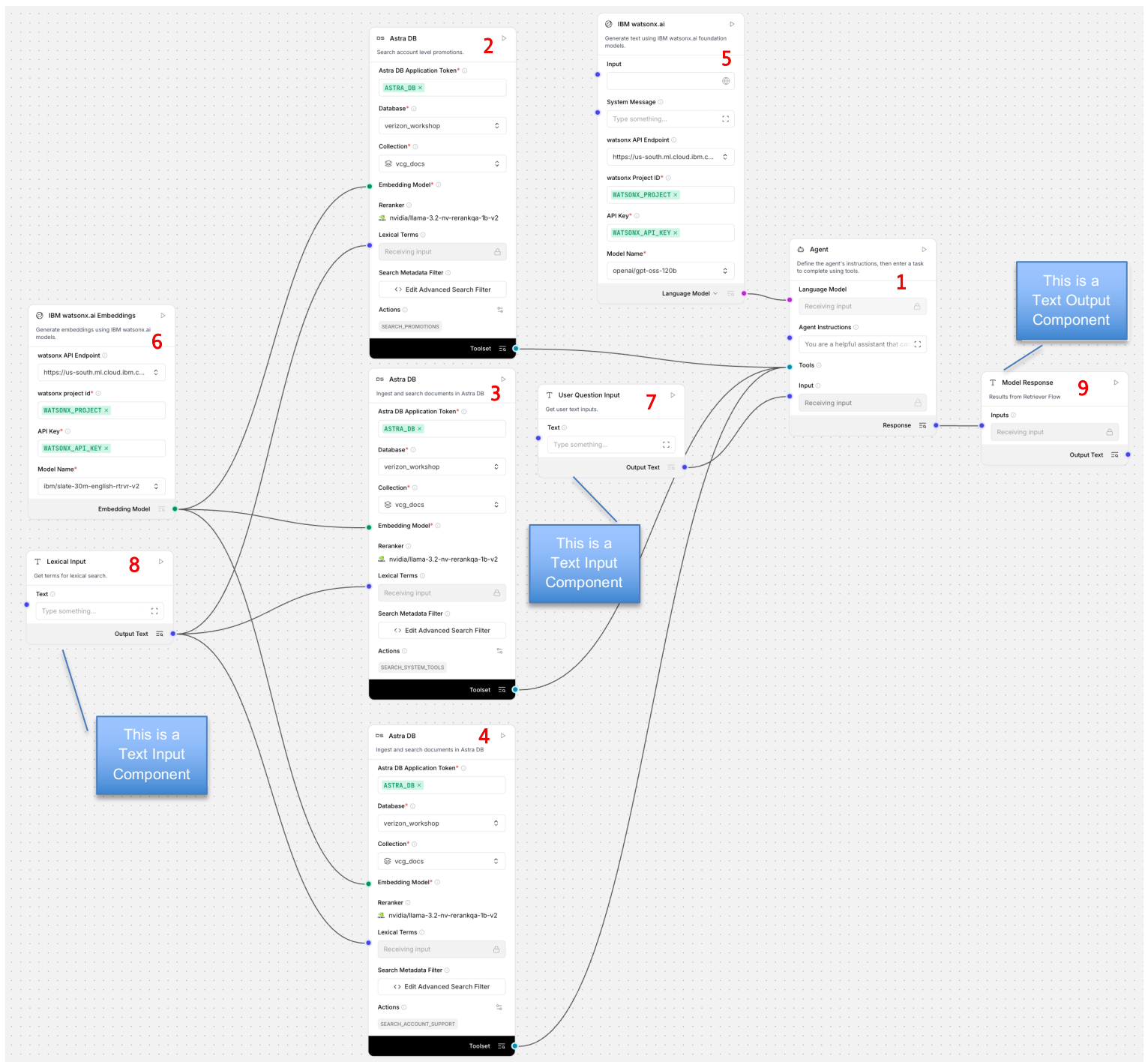
Click the Playground button in the top right to open the chat. Ask some questions of your data. Here are some examples:

- How can I change the features of a customer's account?
- How do I change my customer's calling plan?
- I'm having issues getting my Bluetooth to connect.
- How do I activate a new SIM?
- How do I active an eSIM?



Exercise 4: Agentic Flow with Tool Calling

Tool Calling Agent Overview

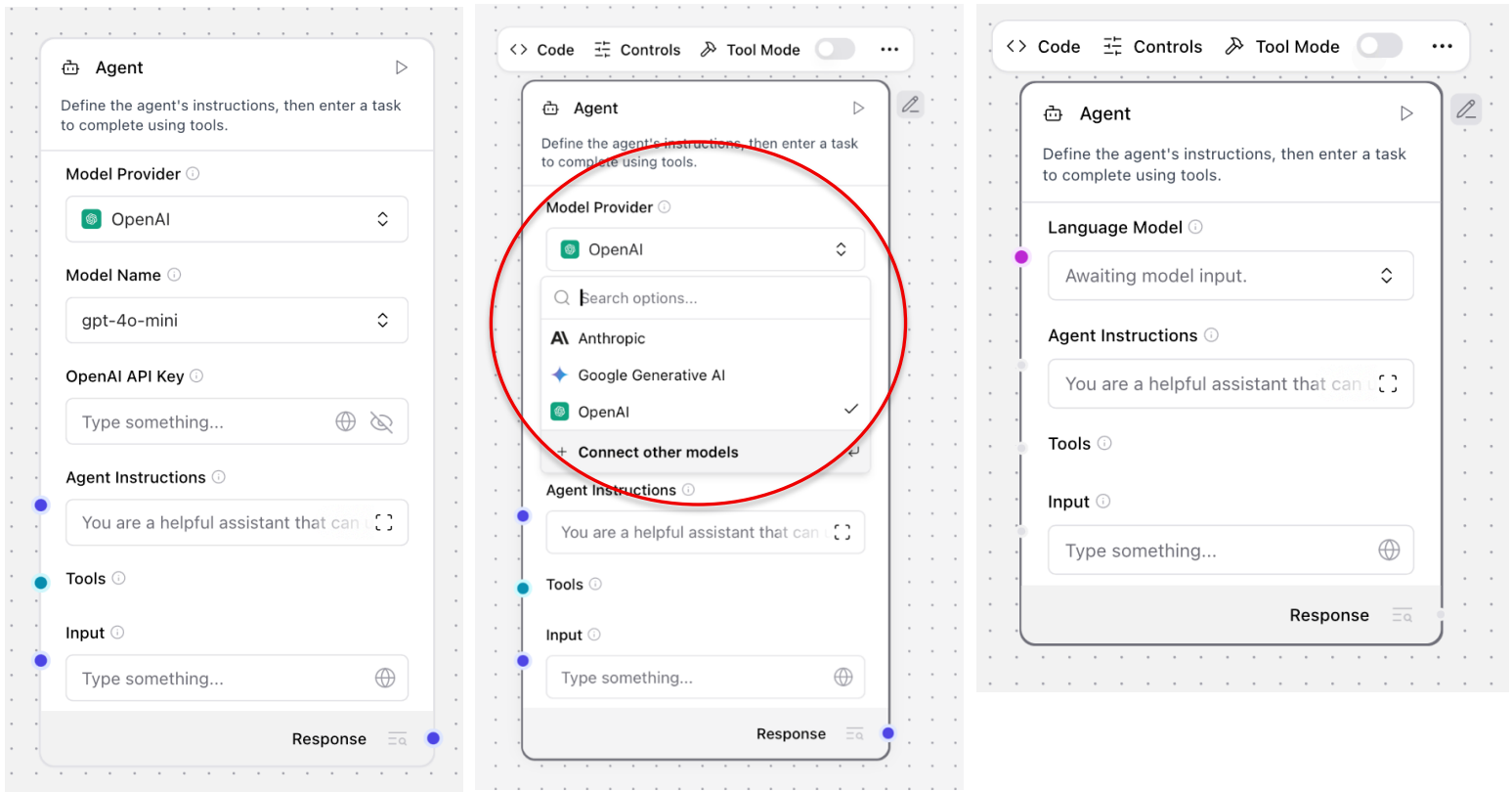


Start with a Blank Flow again from within your Starter Project. Name your flow Tool Calling Agent

1: Agent

At the core of this flow's functionality is the Agent component. This agent orchestrates the flow and chooses which Tool to use to answer the user's question.

When you first drop the Agent onto the canvas, it should look like the screenshot on the left. Click Model Provider, then Connect other models. The Agent component should then look like the screenshot on the right



Fill in the Agent Instructions parameter as follows:

You are a helpful assistant that can use tools to answer questions about Verizon's systems and processes.

Your tools include three vector databases that are optimized to answer questions on specific topics. Those topics are:

- Promotions on products, devices, or plans
- System and Tools provides instructions for Verizon internal systems and device troubleshooting
- Account Support for customer account related questions

Use the tools you have to answer the questions to the best of your ability.

Let's add some additional components before connecting any inputs and outputs.

2/3/4 : Astra DB

All three of these Astra DB components will be *almost* identically configured. Fill in the parameters of all three components as follows.

Astra DB Application Token: **ASTRA_DB**

Database: **verizon_workshop**

Collection: **vcg_docs**

On each of the components, click on the individual component to expose the top toolbar.

Click the toggle button to enable Tool Mode

Click Controls, scroll down to find Search Metadata Filter and click the toggle to Show

Connect the Toolset output to the Tools input of the Agent component **1**

The image is a composite of four screenshots illustrating the configuration and integration of Astra DB components in a workflow.

- Top Left:** A screenshot of the 'Astra DB' configuration panel. The 'Search Metadata Filter' toggle is highlighted with a red circle.
- Top Right:** A screenshot of the 'Astra DB' component in a workflow. The 'Tool Mode' toggle is highlighted with a red circle. The configuration shows 'ASTRA_DB' as the Application Token, 'verizon_workshop' as the Database, and 'vcg_docs' as the Collection.
- Bottom Left:** A screenshot of the 'Search Metadata Filter' component in a workflow. The 'Edit Advanced Search Filter' button is highlighted with a red circle.
- Bottom Right:** A screenshot of the 'Agent' component in a workflow. The 'Tools' input is highlighted with a red circle, and a line connects it to the 'Toolset' output of the 'Search Metadata Filter' component.

2: Astra DB Promotions

Each instance of this Astra DB database component will be configured to only query documents of a particular category. This makes each vector search more accurate and allows the Agent component to choose the appropriate document category to query.

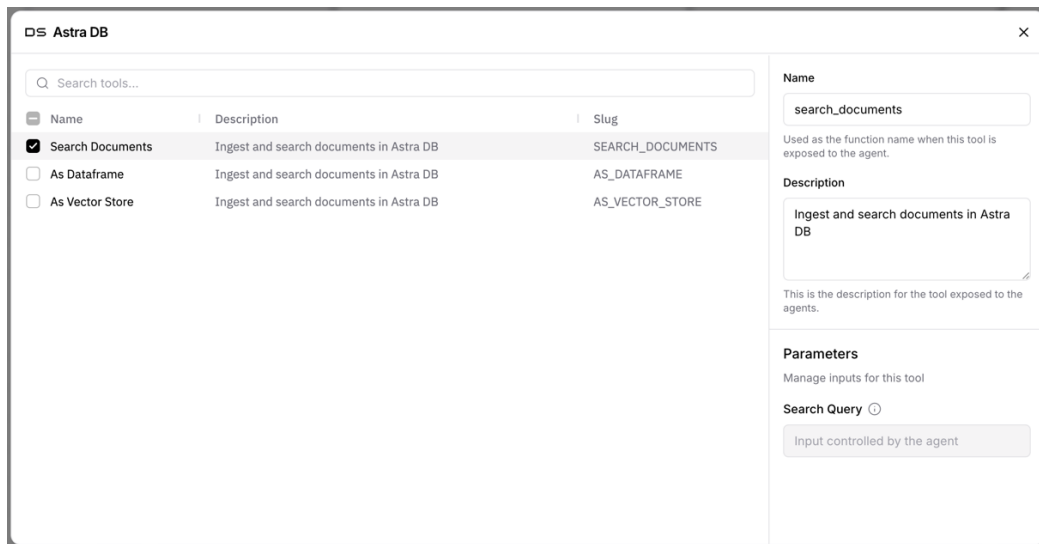
For Astra DB component 2, click Edit Advanced Search Filter (see above) and fill in the filter with this data

```
{
  "category": {
    "$eq": "promotions"
  }
}
```

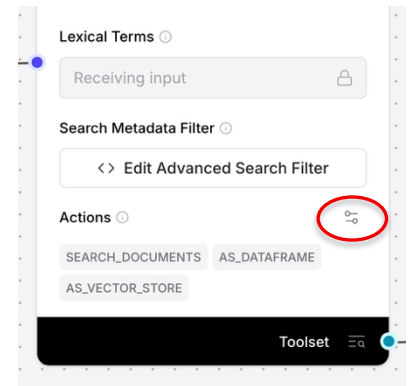
Next we will write the description of the tool’s abilities for the Agent.

Click on the “tennis racket” icon next to Actions to open the dialog.

Uncheck **As DataFrame** and **As Vector Store**. Click on Search Documents:



The screenshot shows the Astra DB configuration interface. On the left, there's a table of tools with columns 'Name', 'Description', and 'Slug'. The 'Search Documents' tool is selected. On the right, the configuration for 'search_documents' is shown, including a name field, a description field, and a parameters section with a 'Search Query' field.



The screenshot shows the 'Edit Advanced Search Filter' dialog. It has sections for 'Lexical Terms', 'Search Metadata Filter', and 'Actions'. The 'Search Metadata Filter' section has a 'tennis racket' icon circled in red. Below it, there are checkboxes for 'SEARCH_DOCUMENTS', 'AS_DATAFRAME', and 'AS_VECTOR_STORE'.

Name: **search_promotions**

Description: **This tool searches product, device or phone plan sales and promotions**

Click the X to close the dialog box.

We will repeat this process with components 3 and 4 with the following parameters:

3: Astra DB System and Tools

Edit Advanced Search Filter

```
{
  "category": {
    "$eq": "system-and-tools"
  }
}
```

Actions (tennis racket)

Name: **search_system_tools**

Description: **Step by step instructions for Verizon internal systems and device troubleshooting**

[Back to Top](#)

4: Astra DB Account Support

Edit Advanced Search Filter

```
{
  "category": {
    "$eq": "account-support"
  }
}
```

Actions (tennis racket)

Name: `search_account_support`

Description: `Search support for your phone account and other customer accounts`

5: IBM watsonx.ai

When you first drag the IBM watsonx.ai component onto the canvas, its output will be Model Response. We need to change it to Language Model. Click on the “down carat” icon and switch the output type to Language Model. See below, and fill in the parameters as follows:

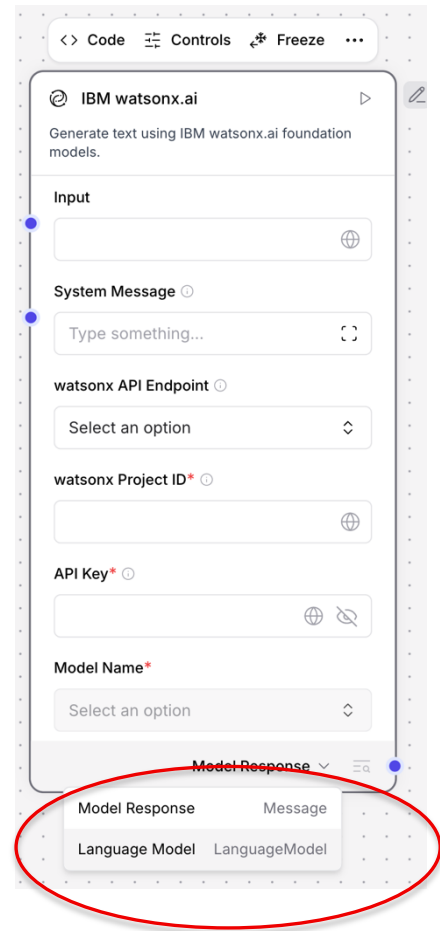
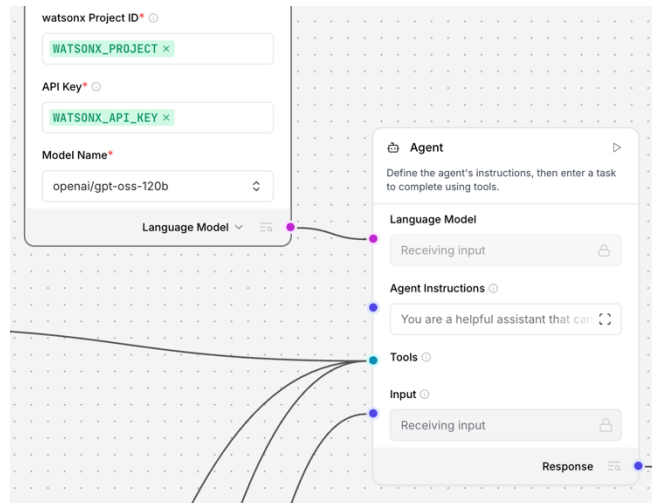
watsonx API Endpoint: `https://us-south.ml.cloud.ibm.com`

watsonx Project ID: `WATSONX_PROJECT`

API Key: `WATSONX_API`

Model Name: `openai/gpt-oss-120b`

Connect the Language Model output to the Language Model input of the Agent component.



6: IBM watsonx.ai Embeddings

Add the IBM watsonx.ai Embedding component and connect its Embedding Model output to all three Astra DB components' Embedding Model input, numbers **2/3/4**. Fill in the parameters as follows:

watsonx API Endpoint: `https://us-south.ml.cloud.ibm.com`

watsonx Project ID: `WATSONX_PROJECT`

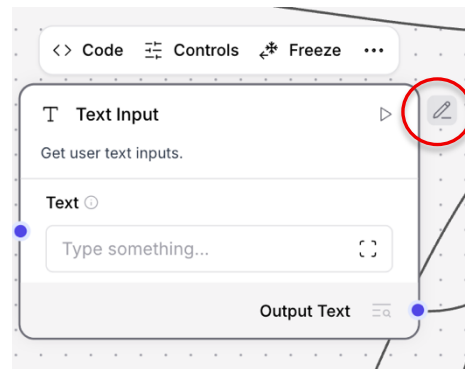
API Key: `WATSONX_API`

Model Name: `ibm/slate-30m-english-rtrvr-v2`

[Back to Top](#)

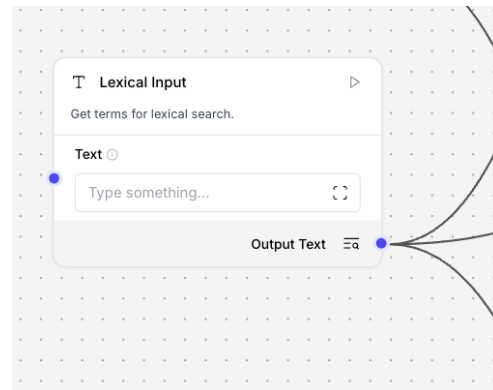
7: Text Input

Drop in a Text Input component and wire its Output Text output to the Input of the Agent component. Click on the Text Input component and select the pencil icon at the top right. Name the component **User Question Input** and click the green check.



8: Text Input

Drop in another Text Input component, and this time wire its Output Text output to the Lexical Terms input of all three Astra DB components, **2/3/4**. Click the pencil icon and rename the component **Lexical Input**.



9: Text Output

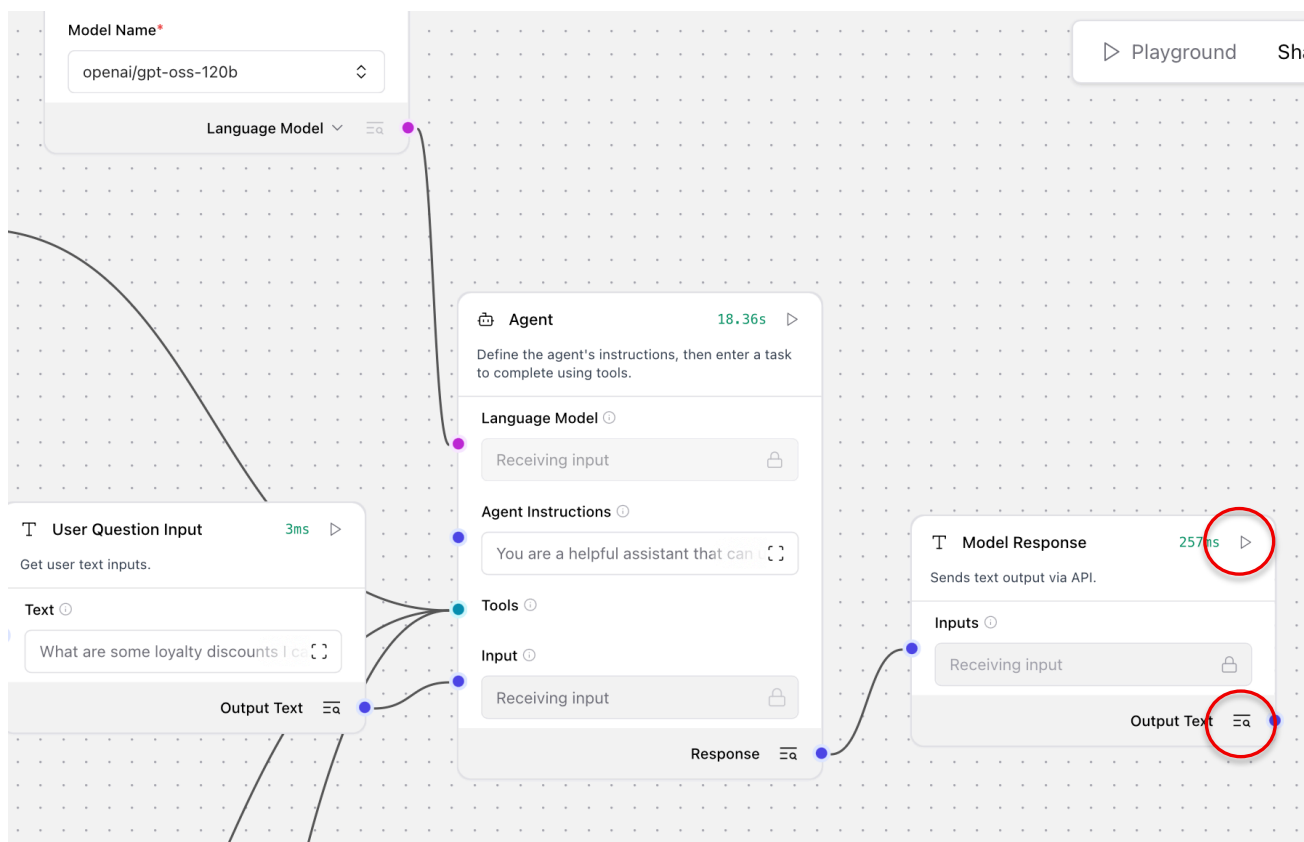
For the final component, add a Text Output component. Wire the Response output of the Agent component to the Input of Text Output. Click the component and then pencil icon and rename the component **Model Response**.

Test your Agent

In the User Question Input component, ask the following question

What are some loyalty discounts I can offer?

Hit the Play icon in the top right of the Model Response component and keep an eye on it. When you see numbers in green appear in the top right, select the Inspect Output button next to Output Text in your Model Response component.



If you see some text like what is shown below, your Tool Calling Agent flow is working properly!

Component Output

Inspect the output of the component below.

Outputs

Logs

Below are the **loyalty-discount programs** that are currently loaded in Verizon's promotion database. I've broken each one out by the key details you'll need when you're deciding which offer to present to a customer.

...

1 Proactive Business-Market Loyalty Discount
["Proactive Loyalty Discounts" – internal promotion ID 362531]

| What the offer is | **10% or 15% off the monthly line price** for up to **12 months** |
|-----|
| Who can receive it | **Business-market customers only** (BE with an ECPD profile). |
| Eligible account size | Accounts with **<= 49 lines**. You can apply the discount to **up to 5 lines per account** (each line gets its own discount). |
| Contract terms that qualify | • Out-of-contract (OOC)
• Month-to-month
• Device-payment agreements
• 1-yr or 2-yr equipment contracts (subsidy) |
| Stackability | Can be combined **only with Bonus-Data offers** (e.g., 5 GB/12 mo or 2 GB/24 mo). It cannot be stacked with any other line-access discounts (8% ECPD profile discount, 22% GPD discount, etc.). |
| How it's presented in the system | The offer appears as a **Featured Action tile** on the MySolutions "Account Inquiry – Overview" screen. |
| Restrictions / exclusions | • Accounts with **> 50 lines**
• Global/Large Enterprise or Public-Sector accounts
• Customers on non-standard pricing
• Consumer, ELEU and corporate non-contracted customers (i.e., BE without ECPD)
• Business Unlimited Start / Pro Tablet / Smartwatch plans |
| Effective dates (internal use only) | **02/22/2023 – 12/31/2023** – do not share dates with customers. |

Quick script you can use

> "Because your account shows a higher propensity to churn, we can lock in a **10% (or 15%) discount** on up to five of your lines for a full year". This is a special loyalty incentive we reserve for valued business customers like you and it can be paired with a bonus-data add-on if you'd like extra data."

Exercise 5: Use Your Agent in the Retriever Flow

Click Starter Project to return to your projects. On the far-right side, hover over your Enriched RAG Retriever flow and click the three dots. Select Duplicate.

Starter Project

FlowsMCP Server

Q Search flows...

Simple Agent

Edited less than a minute ago

...

Enriched RAG Retriever

Edited 9 minutes ago

...

Enriched RAG Ingest

Edited 2 hours ago

...

Edit details

Export

Duplicate

Delete

Starter Project

FlowsMCP Server

Q Search flows...

Tool Calling Agent

Edited 1 minute ago

Enriched RAG Retriever

Edited 21 hours ago

Enriched RAG Ingest

Edited 22 hours ago

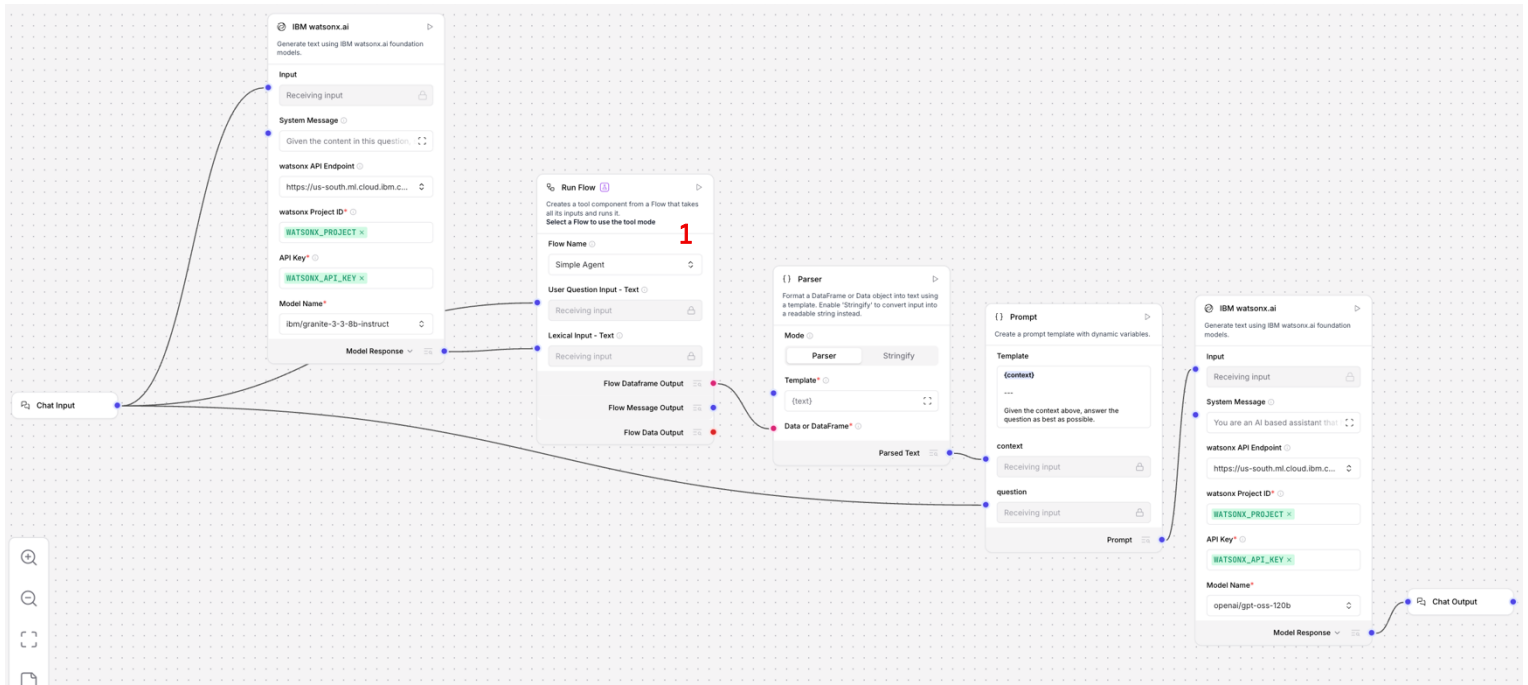
Agent RAG Retriever

Edited 2 days ago

You should now see another flow called Enriched RAG Retriever (1). Click the three dots next to the duplicated flow and select Edit details. Change the name to Agent RAG Retriever.

Make the Description:
Retrieve Verizon specific documents for answering questions.

Agent RAG Retriever Flow Overview



1: Run Flow

Drop the Run Flow component onto the canvas. Click Flow Name and select Tool Calling Agent. After that, delete the Astra DB and IBM watsonx.ai Embeddings components.

You should see the inputs and outputs change to look like component 1 above. As is shown above, wire the output of the Chat Input component into the User Question Input of Run Flow.

Wire the Model Response output of The IBM watsonx.ai component to the Lexical Input of Run Flow. Wire the Flow DataFrame Output to the Data or DataFrame input of Parser.

This retriever flow will now use the Agent you created in the previous exercise.

Test Your Flow

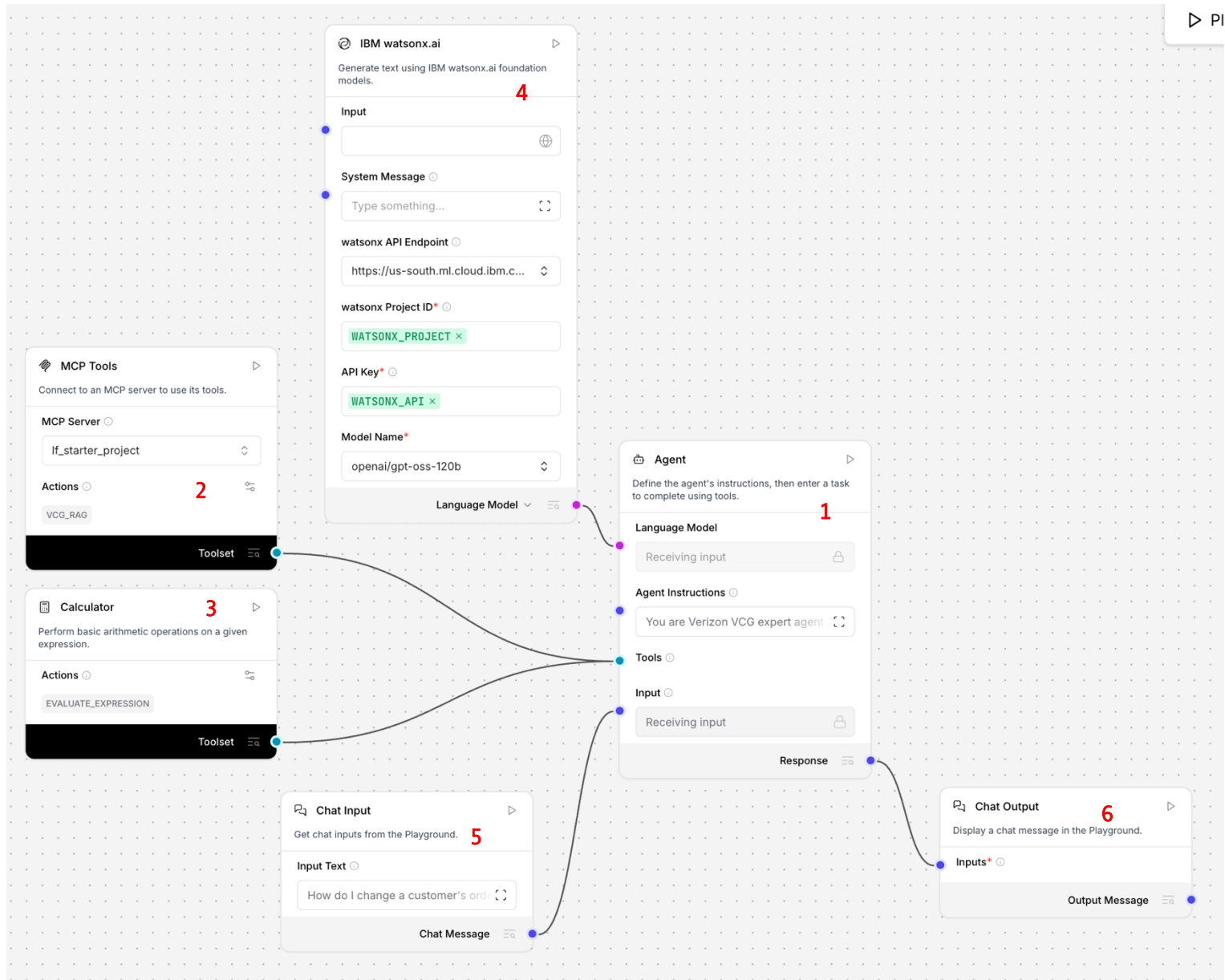
Open the Playground and ask the following question
What is the process to swap devices?

If you get an acceptable answer, your flow is working correctly! [See Appendix A](#) for more questions to ask

Exercise 6: Multi-Agent Flow with MCP Tool

MCP Tool Flow Overview

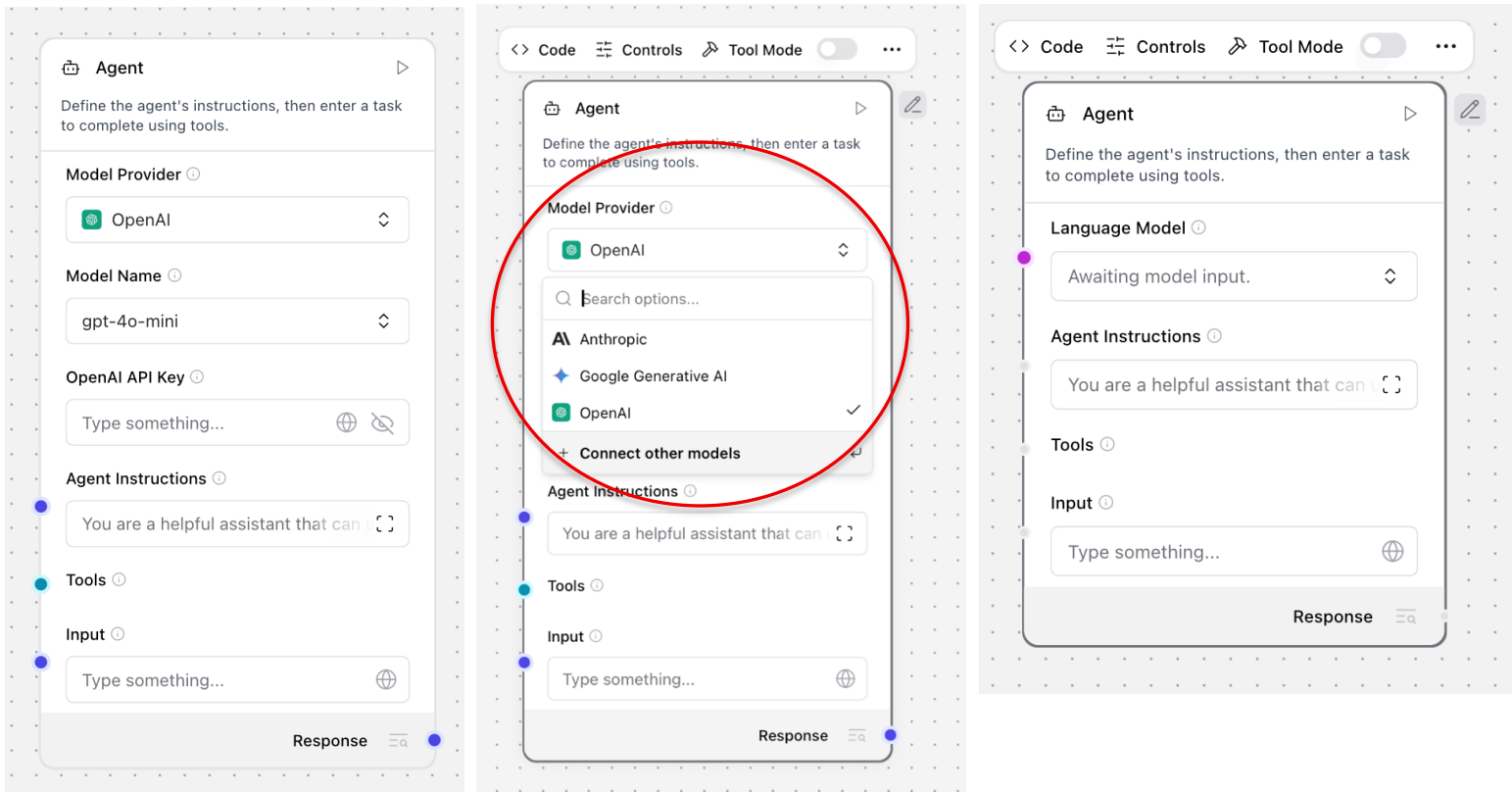
Create another blank flow and name it MCP Multi Agent Chat. Let's build a multi-agent, tool calling workflow!



1: Agent

At the core of this flow's functionality is the Agent component. This agent orchestrates the flow and chooses which Tool to use to answer the user's question.

When you first drop the Agent onto the canvas, it should look like the screenshot on the left. Click Model Provider, then Connect other models. The Agent component should then look like the screenshot on the right



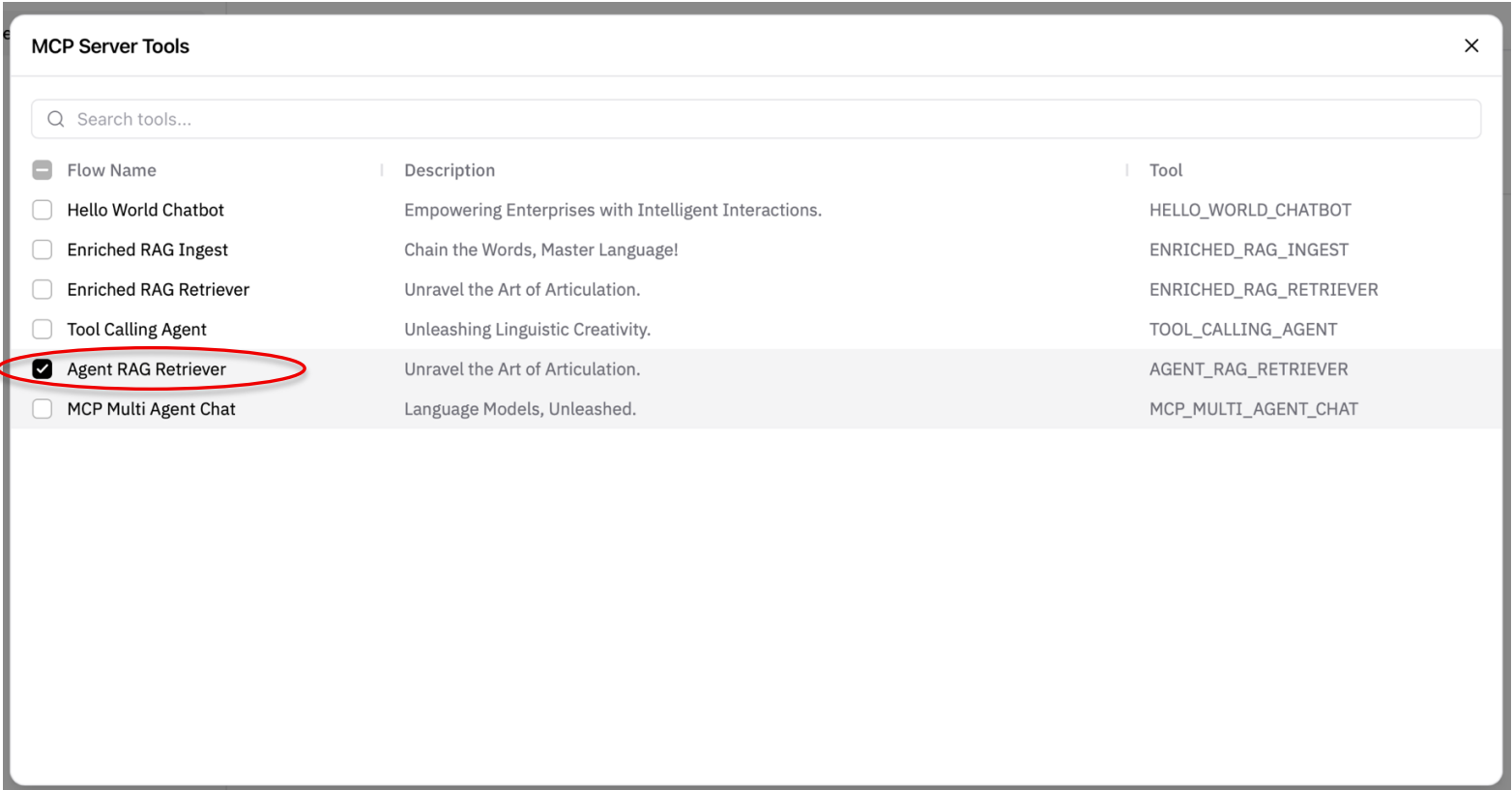
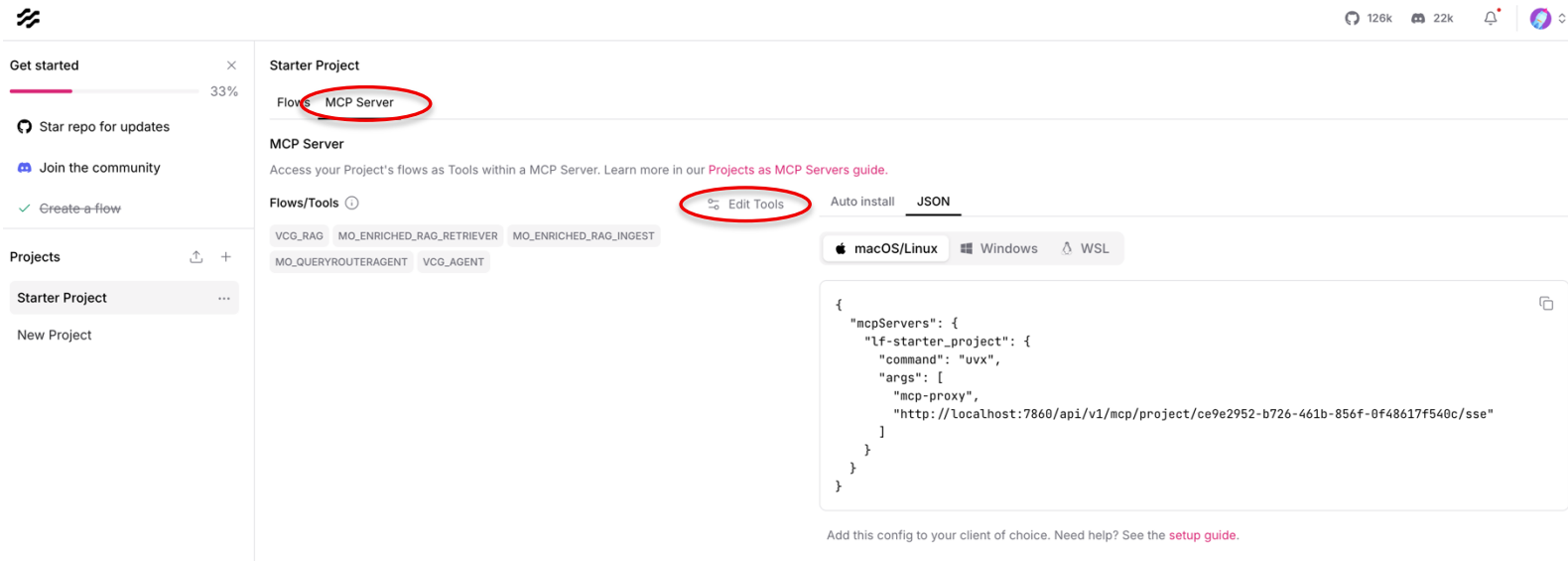
Fill in the Agent Instructions parameter as follows:

You are Verizon VCG expert agent with access to a VCG documentation retrieval tool. Use this tool to answer questions from the user. Only answer questions that you can find VCG documentation to answer, do not make things up or hallucinate. You also have access to a calculator tool to make calculations from the data provided and answer the user question.

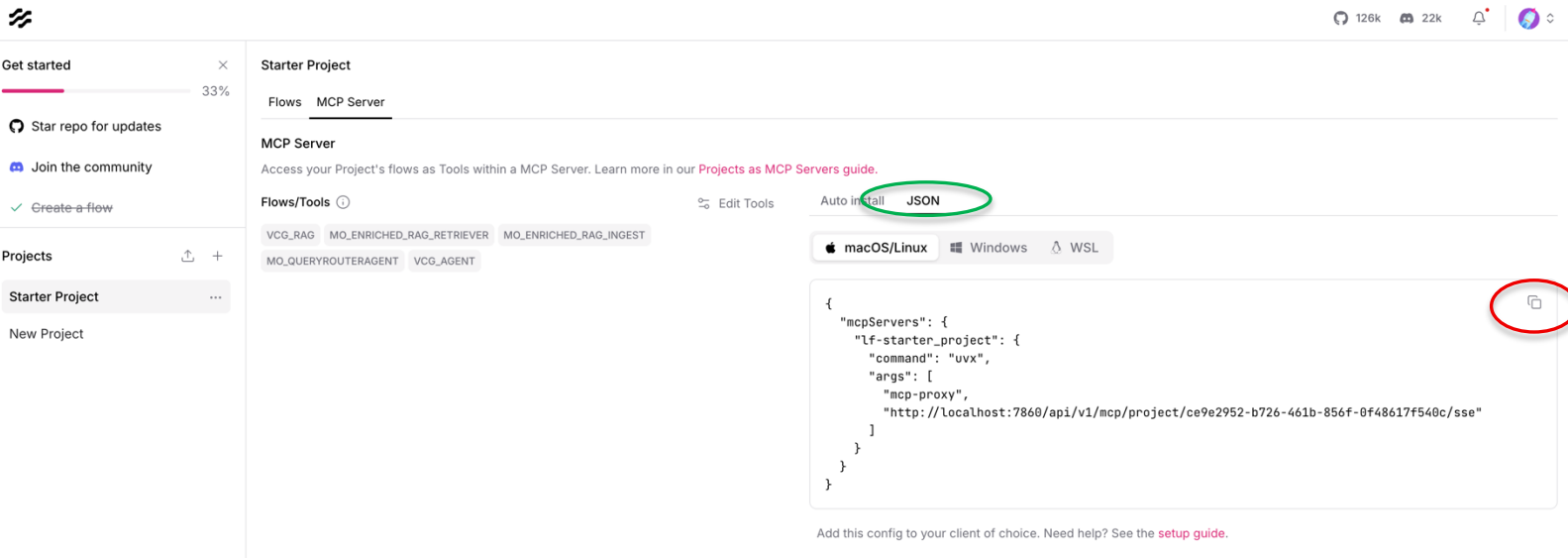
Let's add some additional components before connecting any inputs and outputs.

2: MCP Tools

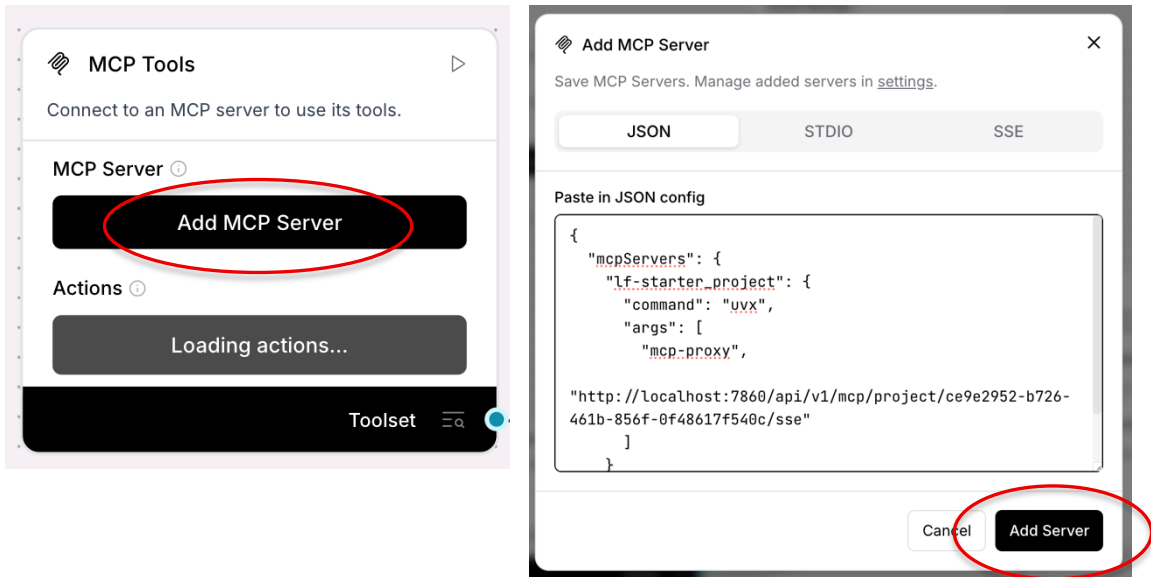
- Drop the MCP Tools component onto the canvas
- Click Starter Project at the top of the page to return to your project folder
- Click MCP Server (next to Flows), then Edit Tools
- Uncheck any checked boxes, and only check the flow Agent RAG Retriever
- Click the X to close the dialog box



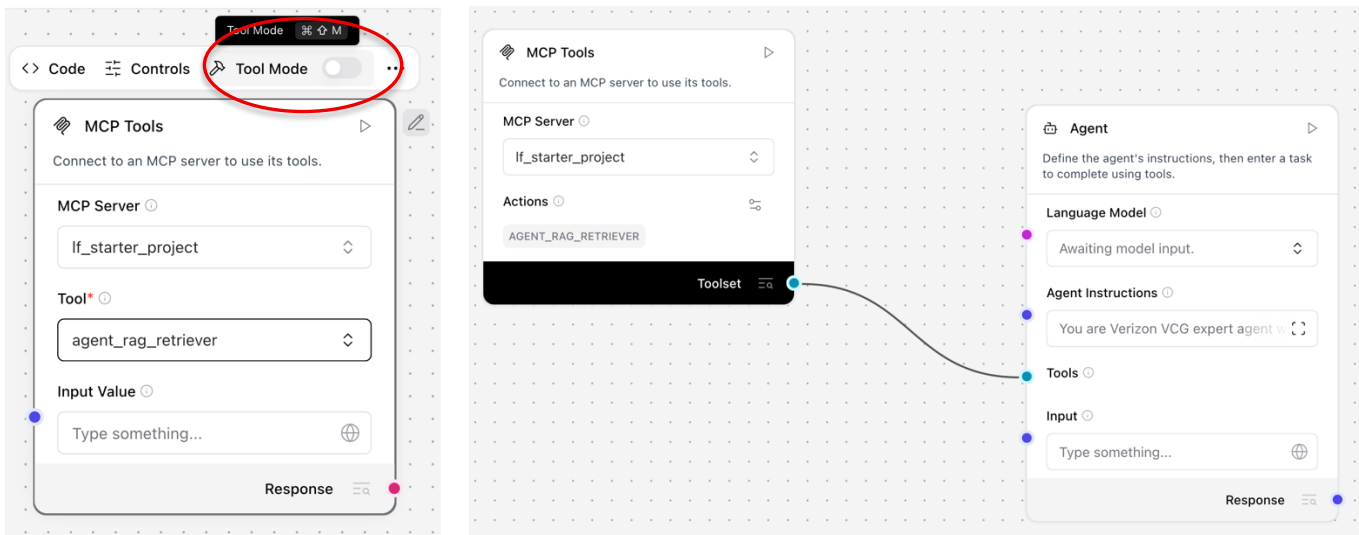
Next click the JSON tab (green circle in screen shot below) for the MCP server configuration
Hit the copy icon and then navigate back to the Flows page, then select your MCP Multi Agent Chat flow



Click Add MCP Server, click JSON, then paste in the server configuration. Click Add Server



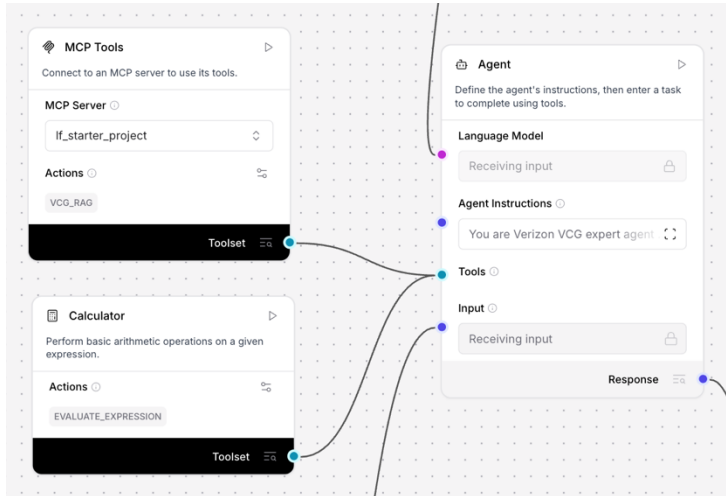
Select your project (there should be only one), select the component's tool
(there should be only one), and put the component into Tool Mode
Connect the MCP Tools Toolset output to the Tools input of the Agent component



[Back to Top](#)

3: Calculator

To give our Agent the ability to perform accurate mathematical functions, we'll supply the Calculator component. Drop it onto the canvas, put the component into Tool Mode, and connect it to the Agent's Tools input



4: IBM watsonx.ai

When you first drag the IBM watsonx.ai component onto the canvas, its output will be Model Response. We need to change it to Language Model. Click on the “down carat” icon and switch the output type to Language Model. See below, and fill in the parameters as follows:

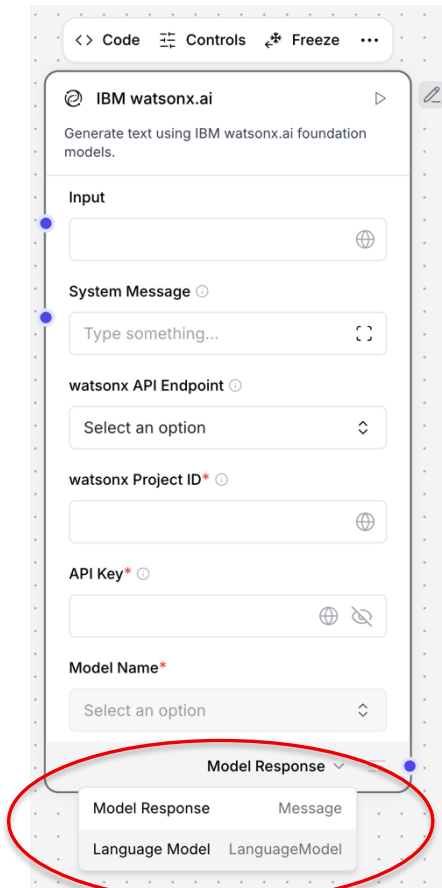
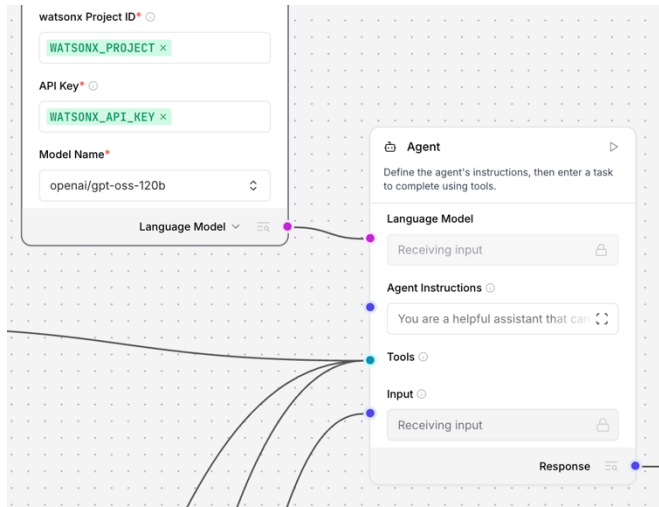
watsonx API Endpoint: <https://us-south.ml.cloud.ibm.com>

watsonx Project ID: **WATSONX_PROJECT**

API Key: **WATSONX_API**

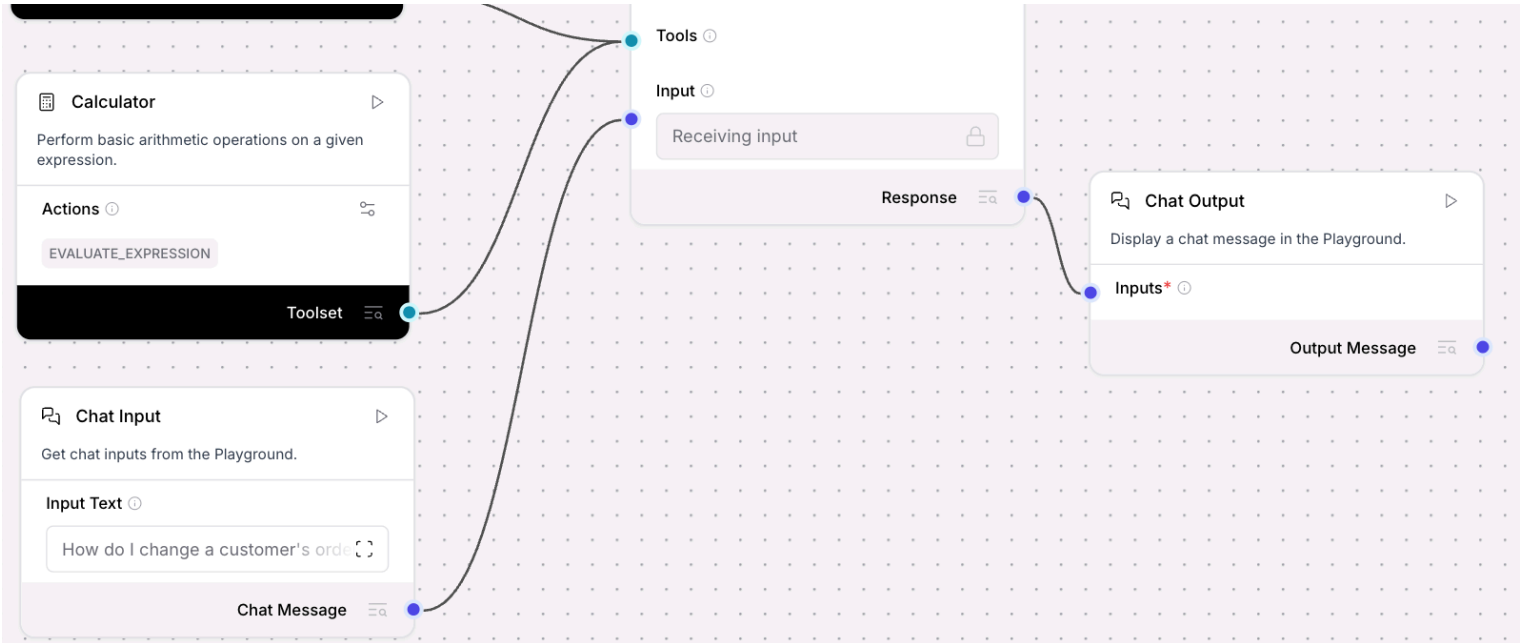
Model Name: **openai/gpt-oss-120b**

Connect the Language Model output to the Language Model input of the Agent component.



5: Chat Input & 6: Chat Output

Wire in the Chat Input and Chat Output as shown below and click Playground in the top right of the page to test your flow.



Appendix A: Document Queries

- How can I change the features of a customer's account?
- How do I change my customer's calling plan?
- I'm having issues getting my Bluetooth to connect.
- How do I activate a new SIM?
- How do I active an eSIM?
- Can I change a device ID?
- What is the process to swap devices?
- How do I activate a new account?
- What are some of the trade-in promotions?
- What are some loyalty discounts I can offer?
- Can I change the address on my customer's account?
- I need to suspend a customer.
- What do I do if my customer suspects they've been a victim of fraud?
- My customer says they have poor service where they work.
- Customer moves to a 'select unlimited' plan on 2025-10-13. Using the Valued Customer Discount terms, what are this-cycle prorated charges and the new monthly total? Include an itemized breakdown and citations.
- For a Visible customer adding Verizon Home Internet with the digital exclusive discount, what's the net monthly bill with/without autopay? Show the delta and cite eligibility/amounts.
- If we add a \$35 activation fee and a \$10 loyalty discount mid-cycle, what is the first bill's total? Itemize base, proration, fee, discount, taxes.

Appendix B: Local Langflow Install

If you need to install Langflow locally on your laptop, follow the below instructions to install dependencies. These steps have been tested on Verizon laptops, but not all environments are the same. Please reach out to bootcamp facilitators or technical assistants for help.

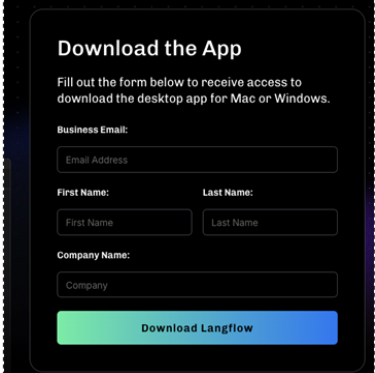
Langflow Desktop

Here you will find the [Internal Verizon Langflow Installation Document](#) that has tips to get you going with Langflow with your Verizon machine.

Get help from your team on Slack and see notes from previous Langflow installs on the Verizon Slack Channel: [#langflow-installation-help](#)

Download and install LangFlow Desktop: <https://www.langflow.org/desktop>

Fill out the form to download the installer and use your Verizon email address

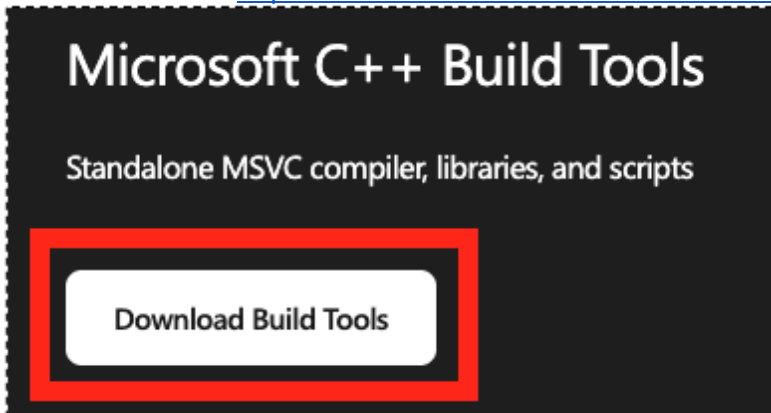
A screenshot of a dark-themed web form titled "Download the App". Below the title, it says "Fill out the form below to receive access to download the desktop app for Mac or Windows." The form contains three input fields: "Business Email:" with a placeholder "Email Address", "First Name:" with a placeholder "First Name", and "Last Name:" with a placeholder "Last Name". Below these is a "Company Name:" field with a placeholder "Company". At the bottom is a large blue button labeled "Download Langflow".

Windows C++ Build Tools

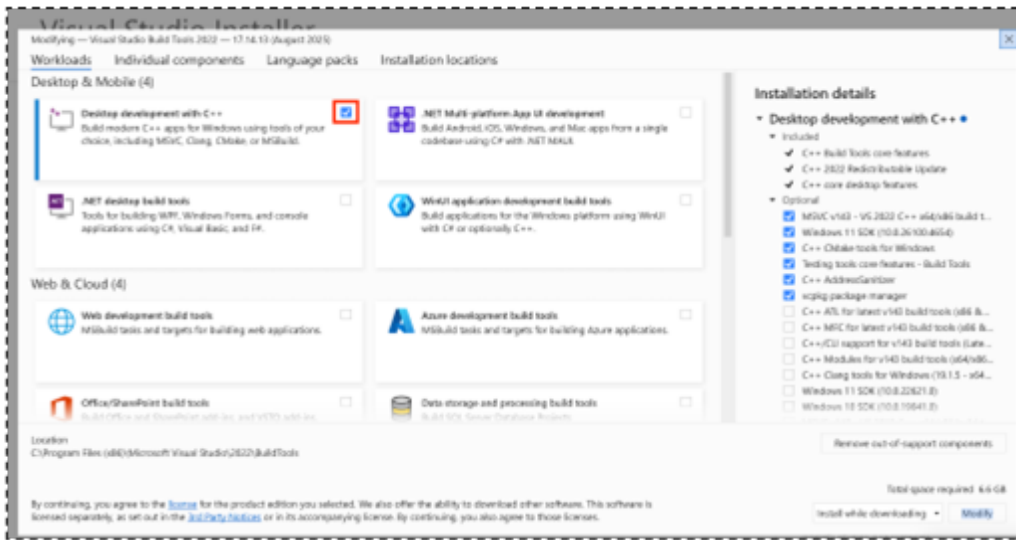
Start this process IMMEDIATELY. (It takes a while!) (If on Mac, skip to the next section.) Note: If you cannot download the following files, please skip to the OSS LangFlow installation steps below.

Microsoft Windows installations of Langflow Desktop require a C++ compiler that may or may not be present on your system. To avoid a **C++ Build Tools Required!** error, perform the following steps.

1. Install build tools: <https://visualstudio.microsoft.com/visual-cpp-build-tools/>



2. After downloading and starting the installer, when you get to this screen, you **MUST select Desktop development with C++:**



3. Continue the installer until Visual Studio completes.
4. Follow the rest of the Langflow Desktop install process from the main document

MacOS xcode

If you do not already have xcode, you may need to install it

Attempt installation from the Mac App Store

1. Open App Store.
2. Search for Xcode.
3. Click Get, then Install.
4. Authenticate with your Apple ID (this may require your MDM-managed account if personal Apple IDs are blocked).
5. ⚠️ If "Install" is greyed out or blocked, perform the below steps
 - a. 🔒 If App Store is blocked, use Apple's Developer download site
 - b. If your organization allows .xip files or developer tools:
 - c. Go to <https://developer.apple.com/download/all/>
 - d. Sign in with an Apple ID.
 - e. Find the latest version of Xcode (.xip) compatible with your macOS.
 - f. Download it, then double-click the .xip file to extract.
 - g. Drag the Xcode.app file into your Applications folder.
6. ⚙️ 5. Accept the Xcode license and install command line tools
 - a. After installation:
 - b. Open Terminal and run:
 - i. `sudo xcodebuild -license`
 - c. Press space to scroll, then type agree to accept.
 - d. Then install the Command Line Tools
 - i. `xcode-select --install`
7. 📦 6. Verify installation
 - a. Run this in Terminal:
 - i. `xcode-select -p`
 - b. You should see:
 - i. `/Applications/Xcode.app/Contents/Developer`

OSS Langflow

If Langflow Desktop installation does not work for you (e.g. network issues, permissions, etc), you can install Open Source Software Langflow (OSS Langflow)

[Back to Top](#)

MacOS

Perform the following in your MacOS terminal

1. `pip install uv`
 - a. Or: `pipx install uv`
2. Navigate / `cd` to your projects directory (wherever Langflow should be installed)
3. `mkdir langflow`
4. `cd langflow`
5. `uv venv env`
6. `source env/bin/activate`
7. `uv pip install langflow`
8. `uv run langflow run`

Open `http://localhost:7860` in your browser

Windows

Install Python (if not already installed)

- Run `cmd` to open a terminal
- `winget install -e --id Python.Python.3.13`

Then, open a NEW terminal (so the PATH gets updated). Then, run:

- `py --version`
 - This should return: Python 3.13.7
- `py -m pip --version`
 - This should return something like:
`pip 25.2 from C:\Users\devin\AppData\Local\Programs\Python\Python313\Lib\site-packages\pip (python 3.13)`
- `py -m pip install uv`

Navigate to your projects directory (wherever Langflow should be installed)

- `mkdir langflow`
- `cd langflow`
- `uv venv lf`
- `lf\Scripts\activate`
- `uv pip install langflow`
- `uv run langflow run`

Note: This takes a while. Be patient. Eventually, you should see this:

```
79e675cb6752_change_datetime_type - Column
'updated_at' has type DATETIME in table 'variable'
WARNING 2025-08-22 11:33:38 - WARNING - 4e5980a44eaa_fix_date_
4e5980a44eaa_fix_date_times_again - Column
'created_at' has type DATETIME in table 'apikey'
WARNING 2025-08-22 11:33:38 - WARNING - 4e5980a44eaa_fix_date_
4e5980a44eaa_fix_date_times_again - Column
'created_at' has type DATETIME in table 'variable'
WARNING 2025-08-22 11:33:38 - WARNING - 4e5980a44eaa_fix_date_
4e5980a44eaa_fix_date_times_again - Column
'updated_at' has type DATETIME in table 'variable'
[08/22/25 11:35:28] INFO 2025-08-22 11:35:28 - INFO - __init__ - Application startup complete.
INFO 2025-08-22 11:35:28 - INFO - __init__ - Uvicorn running on
http://localhost:7860 (Press CTRL+C to quit)
```

Then, open the URL (`http://localhost:7860`) in your browser.